
PROJECT DOCUMENTATION ON

Finance & Billing Analytics

CHAPTER 1: INTRODUCTION

1.1 Introduction

In today's digital era, organizations require efficient financial management systems to handle billing, invoice processing, payment tracking, and financial reporting. Traditional manual methods of maintaining invoices and payment records are time-consuming, prone to human errors, and make data retrieval difficult. To overcome these challenges, organizations are increasingly adopting automated billing and analytics solutions that provide accuracy, security, and real-time insights.

The Finance Billing & Analytics System is a web-based application developed to streamline financial operations by integrating invoice management, payment processing, user authentication, role-based access control, and analytical reporting into a single platform. The system is built using ASP.NET Core MVC, ASP.NET Core Web API, Entity Framework Core, and SQL Server, following modern software engineering principles and industry-standard architecture.

The application enables users to securely manage invoices, record payments, monitor financial performance, and generate meaningful business insights through an interactive dashboard. The system incorporates JWT (JSON Web Token) Authentication and Role-Based Authorization to ensure secure access to resources and protect sensitive financial data.

In addition, the project includes advanced features such as Forgot Password, OTP Verification, Password Reset, Swagger API Documentation, and Dashboard Analytics, making it a comprehensive financial management solution suitable for small and medium-sized organizations.

The architecture of the system follows a layered approach consisting of Presentation Layer, Business Logic Layer, API Layer, and Database Layer. This separation of concerns improves maintainability, scalability, and security while allowing easy future enhancements.

The primary objective of the Finance Billing & Analytics System is to provide a secure, reliable, and user-friendly platform that automates financial operations, minimizes manual effort, improves data accuracy, and supports informed decision-making through analytical reporting.

1.2 Project Overview

The Finance Billing & Analytics System is designed to manage financial activities such as invoice creation, payment tracking, user management, role management, and dashboard reporting. The system provides administrators and authorized users with tools to monitor financial transactions efficiently while maintaining data integrity and security.

The application supports the following major functionalities:

- User Registration and Login
- JWT-Based Authentication
- Role-Based Authorization
- Invoice Management
- Payment Management
- Dashboard Analytics
- Forgot Password Functionality
- OTP Verification
- Password Reset
- Swagger API Documentation
- Secure Database Operations

By integrating these features into a unified platform, the system improves operational efficiency and provides a centralized solution for managing financial records.

1.3 Need for the Project

Many organizations still rely on spreadsheets and manual bookkeeping processes to manage invoices and payments. Such approaches create challenges including:

- Data redundancy and inconsistency
- Delayed payment tracking
- Difficulty in generating reports
- Security vulnerabilities
- Lack of centralized data management
- Limited analytical capabilities

The Finance Billing & Analytics System addresses these issues by providing an automated and secure platform capable of handling financial operations efficiently.

1.4 Scope of the Project

The scope of the project includes:

- Managing invoices and payments electronically
- Maintaining secure user authentication
- Implementing role-based access control
- Providing dashboard analytics and financial summaries
- Supporting password recovery using OTP verification
- Offering API documentation through Swagger
- Enabling future scalability and integration capabilities

The system is intended for organizations seeking a reliable financial management solution with secure access control and reporting features.

1.5 Benefits of the System

The proposed system provides several benefits:

- Reduced manual effort
- Improved accuracy of financial records
- Enhanced data security
- Faster invoice and payment processing
- Better decision-making through analytics
- Centralized data management
- Improved user experience
- Scalable and maintainable architecture

The implementation of this system contributes to efficient financial management and supports organizational growth through digital transformation.

CHAPTER 2: PROJECT OVERVIEW

2.1 Project Overview

The Finance Billing & Analytics System is a comprehensive web-based financial management application developed to automate and streamline billing operations, payment tracking, financial reporting, and user management within an organization. The system provides a centralized platform where users can efficiently manage invoices, monitor payments, analyze financial performance, and access real-time business insights through an interactive dashboard.

The project is developed using ASP.NET Core MVC for the presentation layer, ASP.NET Core Web API for backend services, Entity Framework Core for database operations, and SQL Server as the relational database management system. The application follows a layered architecture that separates presentation, business logic, and data access components, ensuring maintainability, scalability, and security.

The system is designed to eliminate the limitations of traditional manual billing processes by providing automated workflows, secure authentication mechanisms, and analytical reporting capabilities. Through the implementation of JWT (JSON Web Token) Authentication and Role-Based Authorization, the application ensures that only authorized users can access specific functionalities based on their assigned roles.

The project consists of multiple integrated modules that work together to provide complete financial management functionality. These modules include user authentication, invoice management, payment management, dashboard analytics, role management, password recovery, and API documentation.

2.2 Key Features of the System

1. User Authentication Module

The authentication module provides secure access to the application through:

- User Login
- JWT Token Generation
- Session Management
- Role-Based Access Control
- Secure Password Storage using BCrypt

2. Forgot Password and OTP Verification

To enhance security and improve user experience, the system includes:

- Forgot Password Functionality
- OTP Generation
- OTP Verification
- Password Reset Process
- Secure Email-Based Verification

3. Invoice Management Module

This module enables users to manage invoices efficiently by providing:

- Create New Invoice
- View Invoice Details
- Update Invoice Information
- Delete Invoice Records
- Invoice Status Management

4. Payment Management Module

The payment module helps organizations track financial transactions through:

- Payment Recording
- Payment History Tracking
- Invoice-Payment Linking
- Payment Status Monitoring
- Financial Record Maintenance

5. Dashboard Analytics Module

The dashboard provides real-time insights and financial summaries, including:

- Total Invoices
- Total Payments
- Revenue Analytics
- Collection Rate Analysis
- Financial Summary

- Invoice Summary
- Payment Summary

6. Role Management Module

The role management module supports:

- Role Creation
- Role Assignment
- User Permission Management
- Authorization Control

7. API Documentation Module

The project integrates Swagger documentation for:

- API Testing
- Endpoint Documentation
- Request/Response Visualization
- JWT Authorization Testing

2.3 Project Architecture Overview

The system follows a Three-Tier Architecture consisting of:

Presentation Layer (Frontend)

Developed using ASP.NET Core MVC and Razor Views.

Responsibilities:

- User Interface
- Form Validation
- User Interaction
- Dashboard Display

Business Logic Layer (API Layer)

Developed using ASP.NET Core Web API.

Responsibilities:

- Business Rule Processing
- Authentication Handling
- Data Validation
- Request Processing

Data Access Layer

Implemented using Entity Framework Core.

Responsibilities:

- Database Communication
- CRUD Operations
- Query Execution
- Data Persistence

Database Layer

SQL Server Database stores:

- User Information
- Roles
- Invoices
- Payments
- Authentication Data

2.4 Working of the System

The system operates through the following workflow:

1. User enters login credentials.
2. API validates credentials.
3. JWT token is generated upon successful authentication.
4. Authorized users gain access to application modules.
5. Users create and manage invoices.
6. Payments are recorded against invoices.
7. Dashboard aggregates financial information.

8. Reports and analytics are displayed in real time.
9. Users can recover forgotten passwords through OTP verification.

2.5 Advantages of the Proposed System

The proposed Finance Billing & Analytics System offers several advantages:

- Centralized Financial Data Management
- Secure Authentication and Authorization
- Reduced Manual Processing
- Real-Time Financial Analytics
- Improved Accuracy and Reliability
- Easy Maintenance and Scalability
- Industry-Standard Architecture
- Secure API Communication
- Better User Experience
- Future Expansion Capabilities

2.6 Expected Outcomes

After successful implementation, the system is expected to:

- Improve billing efficiency.
- Reduce operational errors.
- Enhance financial visibility.
- Strengthen data security.
- Simplify invoice and payment management.
- Support business decision-making through analytics.
- Provide a scalable foundation for future enhancements.

The Finance Billing & Analytics System therefore serves as a complete solution for managing financial transactions, monitoring business performance, and ensuring secure access to organizational financial data.

CHAPTER 3: PROBLEM STATEMENT

3.1 Problem Statement

In modern organizations, financial operations such as invoice generation, payment tracking, billing management, and financial reporting play a critical role in ensuring smooth business processes. Many small and medium-sized organizations still rely on traditional methods such as spreadsheets, paper records, or disconnected software systems to manage their financial transactions. These manual approaches often result in inefficiencies, inaccuracies, security risks, and difficulties in generating real-time business insights.

Managing invoices and payments manually increases the chances of data duplication, calculation errors, delayed payment tracking, and loss of important financial information. Furthermore, organizations face challenges in maintaining secure access control, monitoring financial performance, and generating analytical reports required for informed decision-making.

Another major issue is the lack of a centralized system that integrates user management, billing operations, payment records, authentication, authorization, and reporting functionalities into a single platform. Without proper security mechanisms, sensitive financial information becomes vulnerable to unauthorized access and data breaches.

Additionally, users often experience difficulties in recovering accounts when passwords are forgotten, leading to operational delays and reduced productivity. Traditional systems also lack automated password recovery mechanisms and secure authentication methods.

Therefore, there is a need for a secure, scalable, and user-friendly financial management system that automates billing processes, manages invoices and payments efficiently, provides real-time analytics, and ensures secure access through modern authentication and authorization techniques.

The proposed Finance Billing & Analytics System addresses these challenges by providing a centralized web-based platform that integrates invoice management, payment tracking, dashboard analytics, role-based access control, JWT authentication, OTP-based password recovery, and API-driven architecture. The system improves operational efficiency, enhances data security, reduces manual effort, and enables organizations to make data-driven financial decisions.

3.2 Existing System Problems.

- Time-consuming payment tracking processes.
- Lack of centralized financial data management.
- Increased possibility of human errors.
- Difficulty in generating accurate reports.
- Poor data security and access control.
- Limited analytical and dashboard capabilities.
- No role-based authorization mechanism.
- Absence of secure password recovery functionality.
- Difficulty in monitoring organizational financial performance.

3.3 Proposed Solution

To overcome the limitations of traditional systems, the proposed Finance Billing & Analytics System provides:

- Secure User Authentication and Authorization.
- JWT-Based API Security.
- Invoice Management Module.
- Payment Management Module.
- Dashboard Analytics and Reporting.
- Role-Based Access Control.
- Forgot Password and OTP Verification.
- Secure SQL Server Database Integration.
- Swagger API Documentation.
- Scalable and Maintainable Architecture.

The system automates financial processes and provides a secure environment for managing billing operations while delivering real-time insights into organizational financial performance.

3.4 Objectives Derived from the Problem

The primary objectives of the proposed solution are:

1. To automate invoice and payment management processes.
2. To reduce manual effort and operational errors.
3. To provide secure authentication and authorization mechanisms.
4. To centralize financial information in a single database.
5. To improve financial visibility through dashboard analytics.
6. To enable secure password recovery through OTP verification.
7. To provide well-documented APIs for future integration.
8. To develop a scalable and maintainable enterprise-level application.

By addressing these objectives, the Finance Billing & Analytics System provides a modern solution for efficient financial management and business analytics.

CHAPTER 4: OBJECTIVES

4.1 Primary Objective

The primary objective of the Finance Billing & Analytics System is to develop a secure, efficient, and user-friendly web application that automates financial operations such as invoice management, payment tracking, user authentication, and financial analytics. The system aims to reduce manual effort, improve data accuracy, and provide real-time insights into organizational financial performance.

4.2 Specific Objectives

1. Automate Billing Operations

To eliminate manual invoice processing and provide a systematic approach for creating, updating, viewing, and managing invoices.

2. Efficient Payment Management

To maintain accurate payment records, track payment history, and monitor financial transactions associated with invoices.

3. Secure User Authentication

To implement a robust authentication mechanism using JWT (JSON Web Token) that ensures only authorized users can access the system.

4. Role-Based Authorization

To provide different levels of access permissions based on user roles, thereby enhancing system security and control.

5. Dashboard Analytics

To provide real-time financial summaries, invoice statistics, payment reports, and collection rate analysis through an interactive dashboard.

6. Password Recovery Mechanism

To enable users to securely recover their accounts through OTP-based password reset functionality.

7. Centralized Data Management

To maintain all financial information in a centralized SQL Server database for consistency, reliability, and easy retrieval.

8. API-Based Architecture

To develop RESTful APIs that facilitate communication between the frontend and backend while supporting future integrations.

9. Improve Data Security

To protect sensitive financial information through encryption, authentication, authorization, and secure database operations.

10. Scalability and Maintainability

To design the system using industry-standard architecture patterns that support future enhancements and easy maintenance.

4.3 Expected Outcomes

Upon successful implementation, the system will:

- Reduce manual work and paperwork.
- Improve billing and payment processing efficiency.
- Enhance financial data security.
- Provide accurate analytical reports.
- Improve decision-making through real-time insights.
- Offer a scalable and maintainable software solution.

CHAPTER 5: TECHNOLOGY STACK

5.1 Introduction

The Finance Billing & Analytics System is developed using modern Microsoft technologies and follows industry-standard software development practices. The selected technology stack ensures scalability, security, maintainability, and high performance.

5.2 Frontend Technologies

ASP.NET Core MVC

ASP.NET Core MVC is used for developing the user interface and handling user interactions.

Features:

- Separation of Concerns (MVC Architecture)
- Razor View Engine
- Strongly Typed Views
- Form Validation
- Responsive User Interface

Razor Views

Razor syntax is used to dynamically generate HTML pages and bind server-side data with the user interface.

Bootstrap

Bootstrap is used for responsive design and modern user interface development.

Benefits:

- Mobile Responsive Design
- Reusable Components
- Faster UI Development
- Consistent User Experience

5.3 Backend Technologies

ASP.NET Core Web API

ASP.NET Core Web API serves as the backend service layer responsible for processing requests and implementing business logic.

Features:

- RESTful Architecture
- JSON Data Exchange
- High Performance
- Easy Integration
- Secure API Communication

5.4 Database Technology

Microsoft SQL Server

SQL Server is used as the primary database management system.

Features:

- Relational Database Structure
- High Security
- Data Integrity
- Backup and Recovery Support
- Efficient Query Processing

5.5 ORM Framework

Entity Framework Core (EF Core)

Entity Framework Core simplifies database operations through object-relational mapping.

Advantages:

- Code-First Development
- Database Migrations
- LINQ Queries

5.6 Authentication Technology

JWT (JSON Web Token)

JWT is used to secure API endpoints and authenticate users.

Benefits:

- Stateless Authentication
- Secure Token-Based Access
- Improved Scalability
- Enhanced Security

BCrypt Password Hashing

BCrypt is used to securely store user passwords.

Benefits:

- Password Encryption
- Protection Against Brute Force Attacks
- Industry-Standard Security Practice

5.7 API Documentation

Swagger

Swagger is integrated for API documentation and testing.

Features:

- Interactive API Testing
- Endpoint Documentation
- Request and Response Examples
- JWT Authorization Support

5.8 Development Tools

Microsoft Visual Studio 2026

Used as the primary development environment.

Features:

- Integrated Debugging
- IntelliSense Support
- Code Refactoring
- Database Tools

SQL Server Management Studio (SSMS)

Used for database design and management.

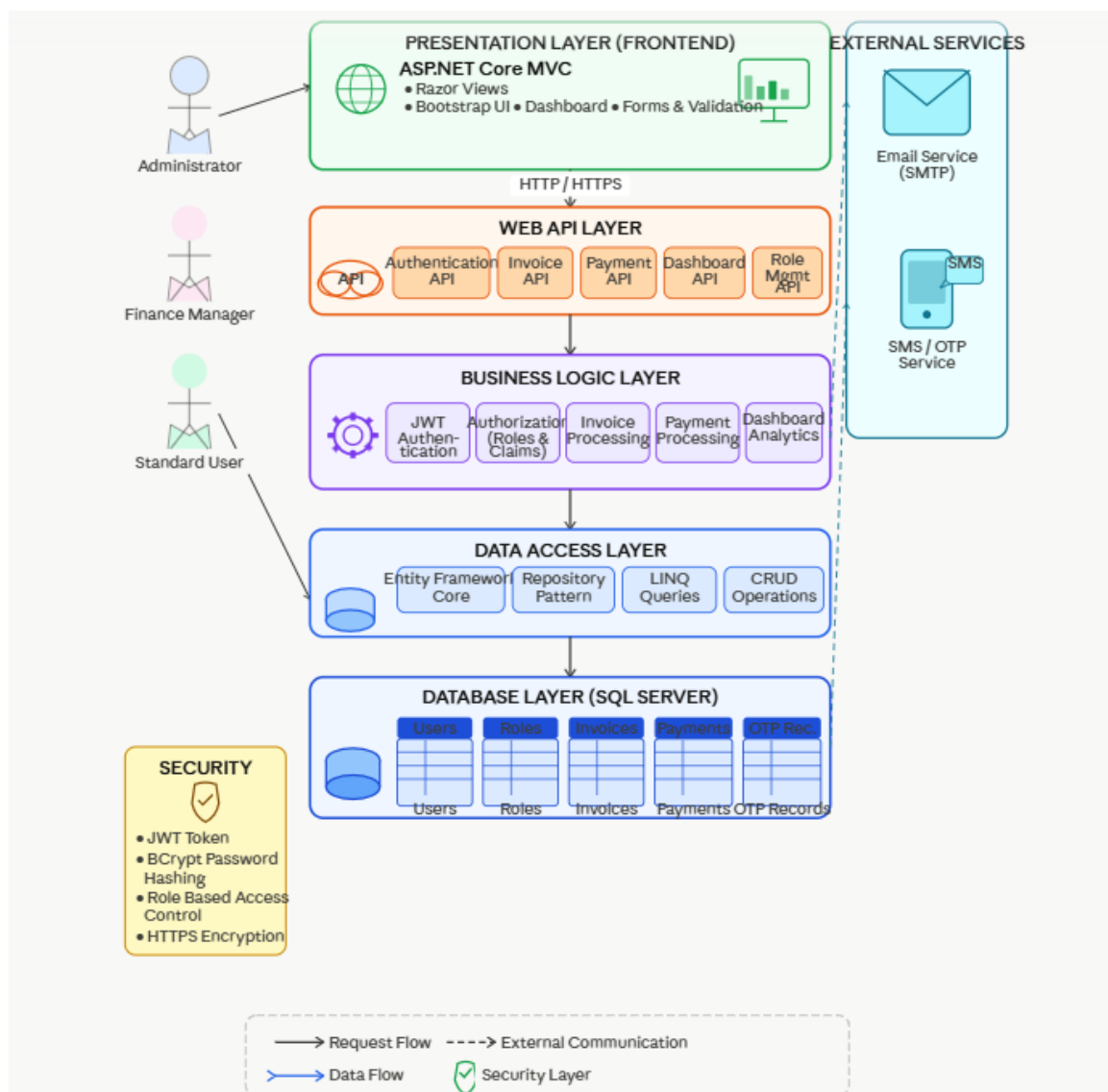
The selected technology stack provides a robust foundation for developing a secure, scalable, and enterprise-level Finance Billing & Analytics System.

CHAPTER 6: SYSTEM ARCHITECTURE DIAGRAM

6.1 System Architecture

The Finance Billing & Analytics System follows a Three-Tier Architecture consisting of the Presentation Layer, Business Logic Layer, and Database Layer. This architecture ensures separation of concerns, maintainability, scalability, and security.

System Architecture Diagram



6.2 Architecture Components

1. Client Layer

The client layer consists of users who interact with the system through a web browser. Different users have different access privileges based on their assigned roles.

Users include:

- Administrator
- Finance Manager
- Standard User

2. Presentation Layer

The presentation layer is developed using ASP.NET Core MVC and Razor Views. It provides the graphical user interface through which users perform operations such as login, invoice management, payment tracking, and dashboard monitoring.

Responsibilities:

- User Interface Rendering
- Data Display
- Form Submission
- Input Validation
- Dashboard Visualization

3. API Layer

The API layer acts as a communication bridge between the frontend and backend services. It exposes RESTful endpoints that process requests and return JSON responses.

Responsibilities:

- Request Processing
- Authentication APIs
- Invoice APIs
- Payment APIs
- Dashboard APIs
- Role Management APIs

4. Business Logic Layer

This layer contains all business rules and application logic.

Responsibilities:

- Invoice Validation
- Payment Processing
- Dashboard Calculations
- OTP Verification
- JWT Token Generation
- Authorization Management

5. Data Access Layer

The Data Access Layer uses Entity Framework Core to communicate with SQL Server.

Responsibilities:

- Database Connectivity
- CRUD Operations
- Query Execution
- Data Persistence

6. Database Layer

SQL Server stores all application data.

Main Database Tables:

- Users
- Roles
- Invoices
- Payments
- OTP Verification Records

6.3 Request Processing Flow

The request flow in the system is as follows:

1. User sends request from browser.
2. MVC Controller receives request.

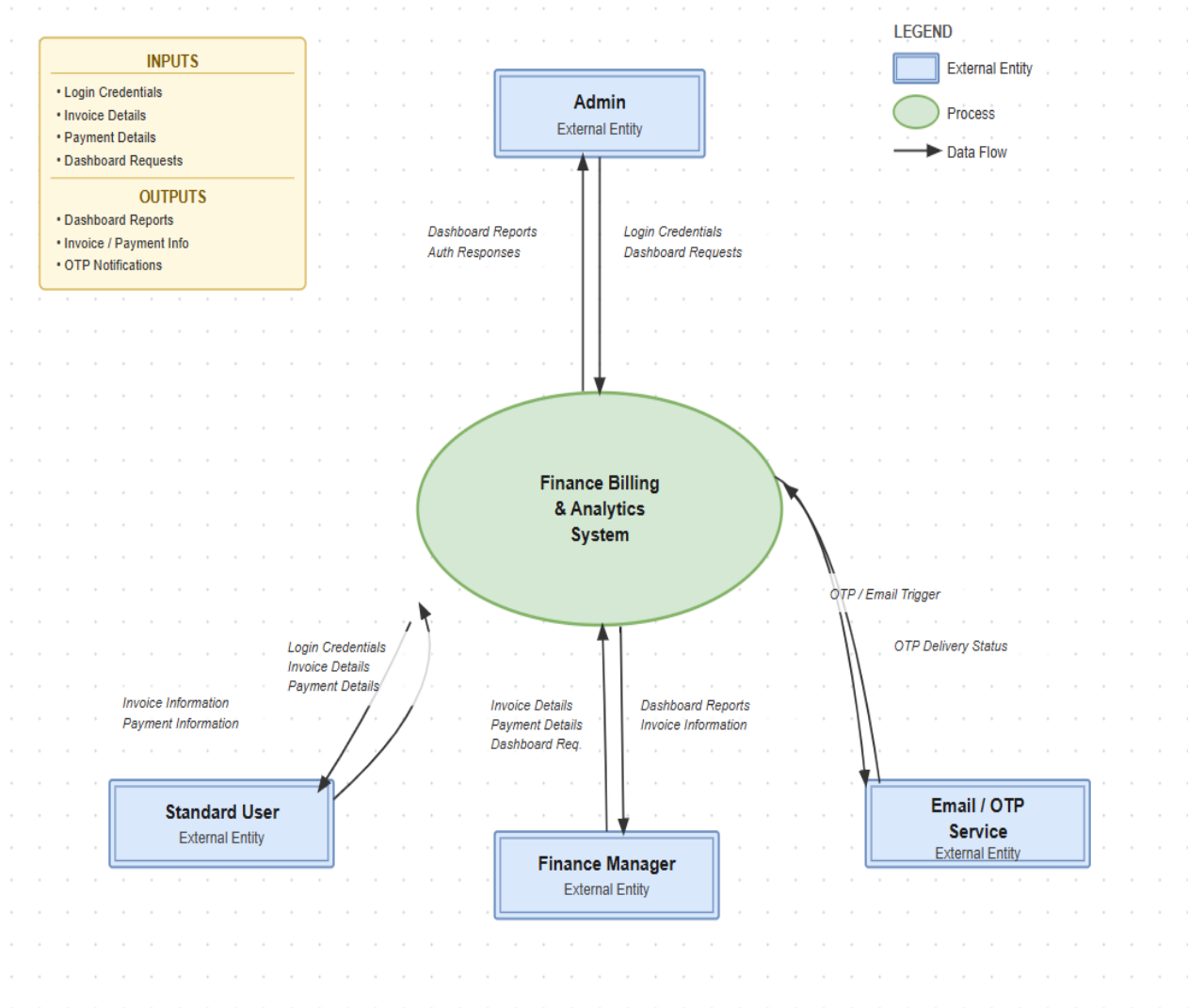
3. Controller calls Web API endpoint.
4. API validates JWT token.
5. Business logic is executed.
6. Entity Framework Core accesses SQL Server.
7. Database returns data.
8. API returns JSON response.
9. MVC displays data to user.

This architecture provides high security, easy maintenance, and scalability while following industry-standard software development practices.

CHAPTER 7: DATA FLOW DIAGRAM

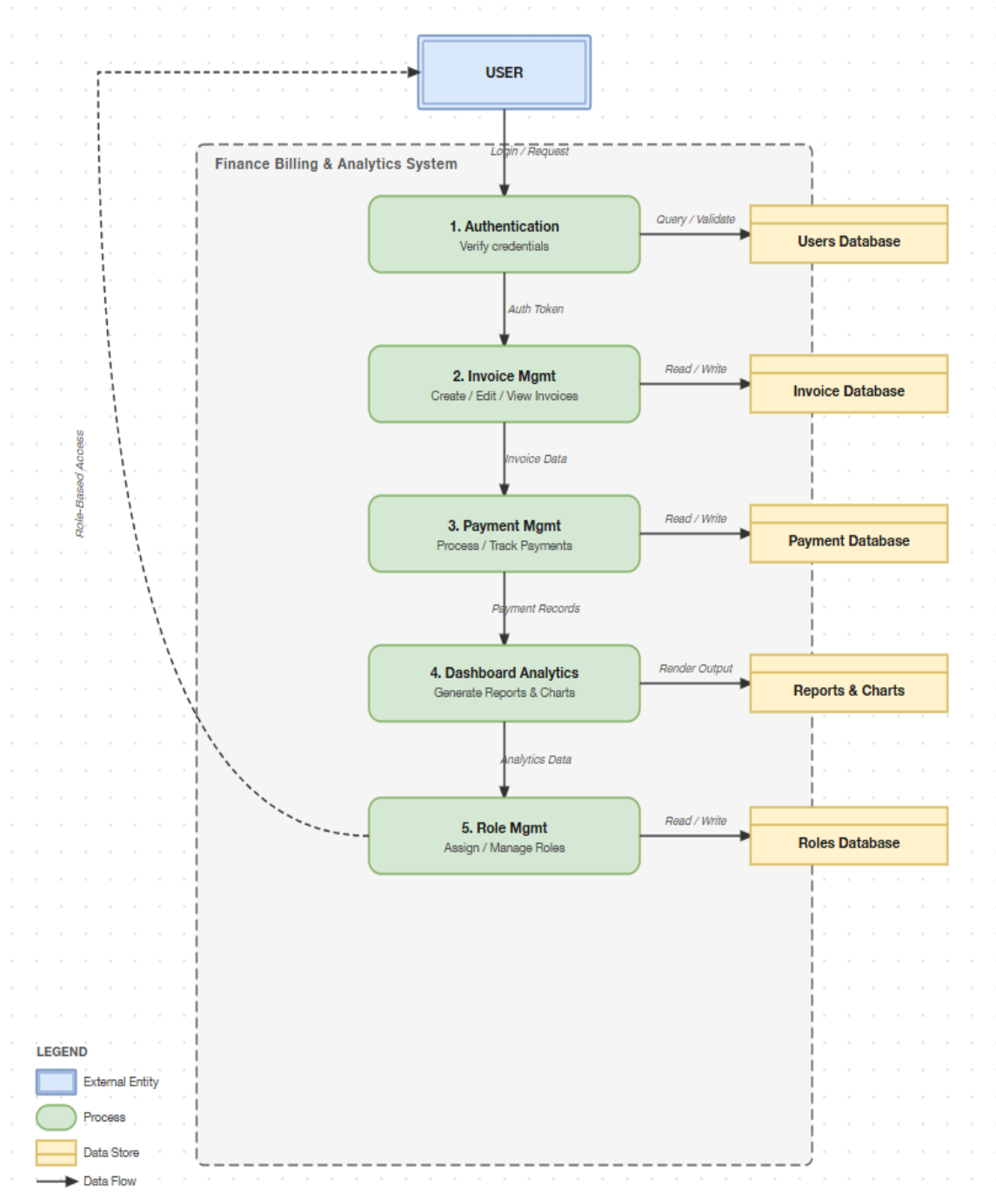
DFD LEVEL 0 (Context Diagram)

Shows the entire system as a single process.



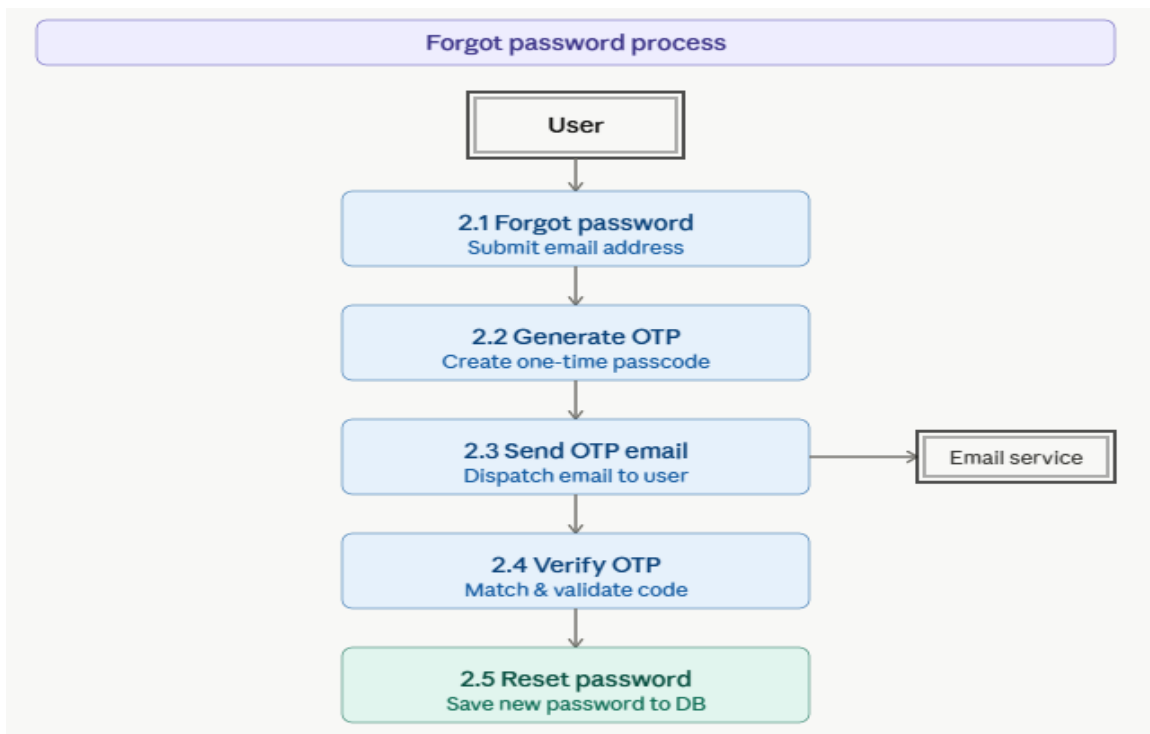
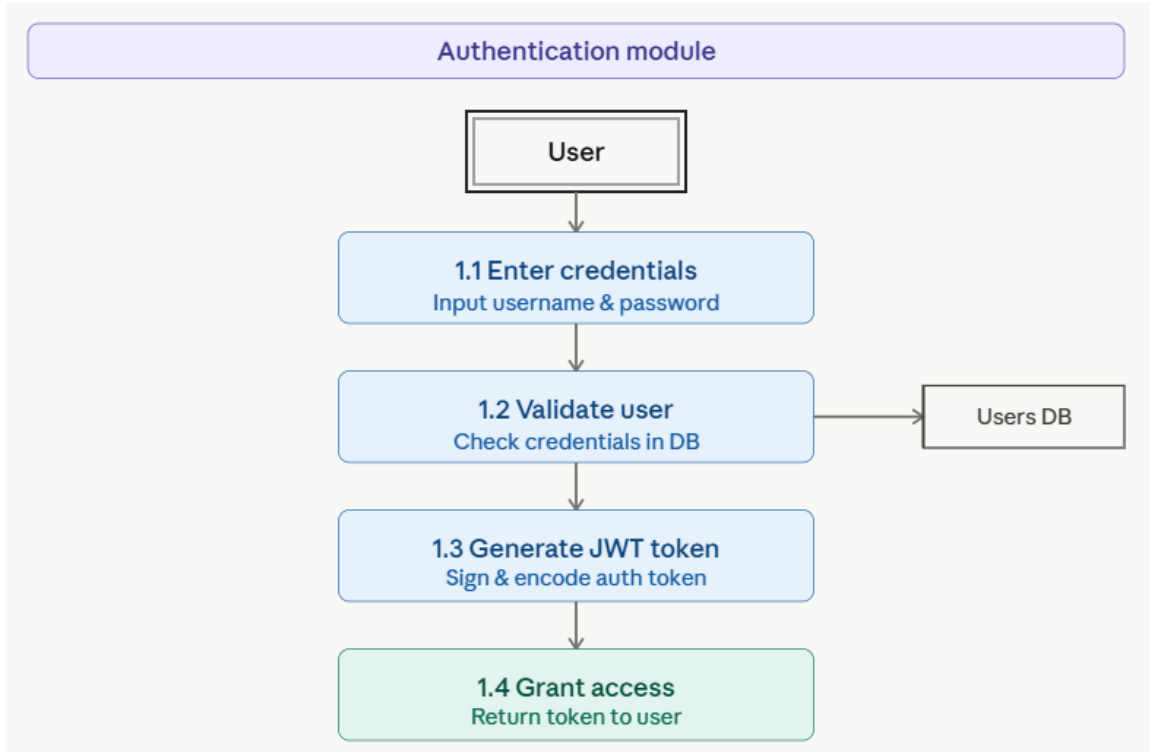
DFD LEVEL 1

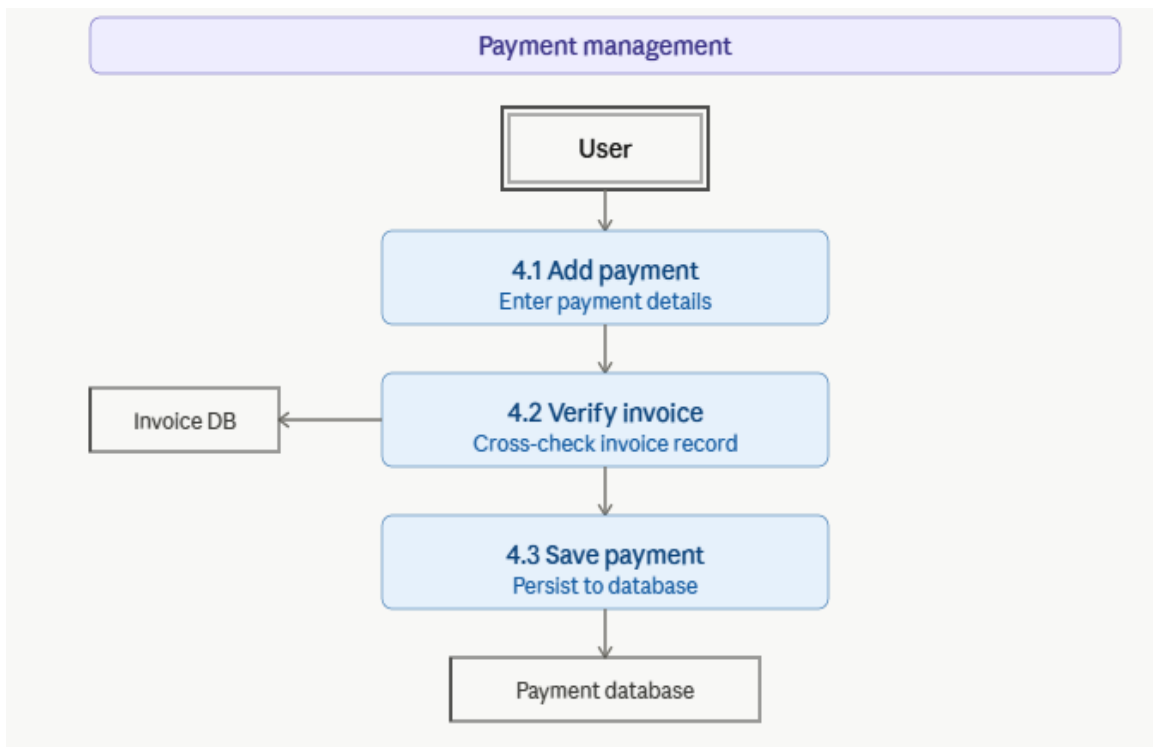
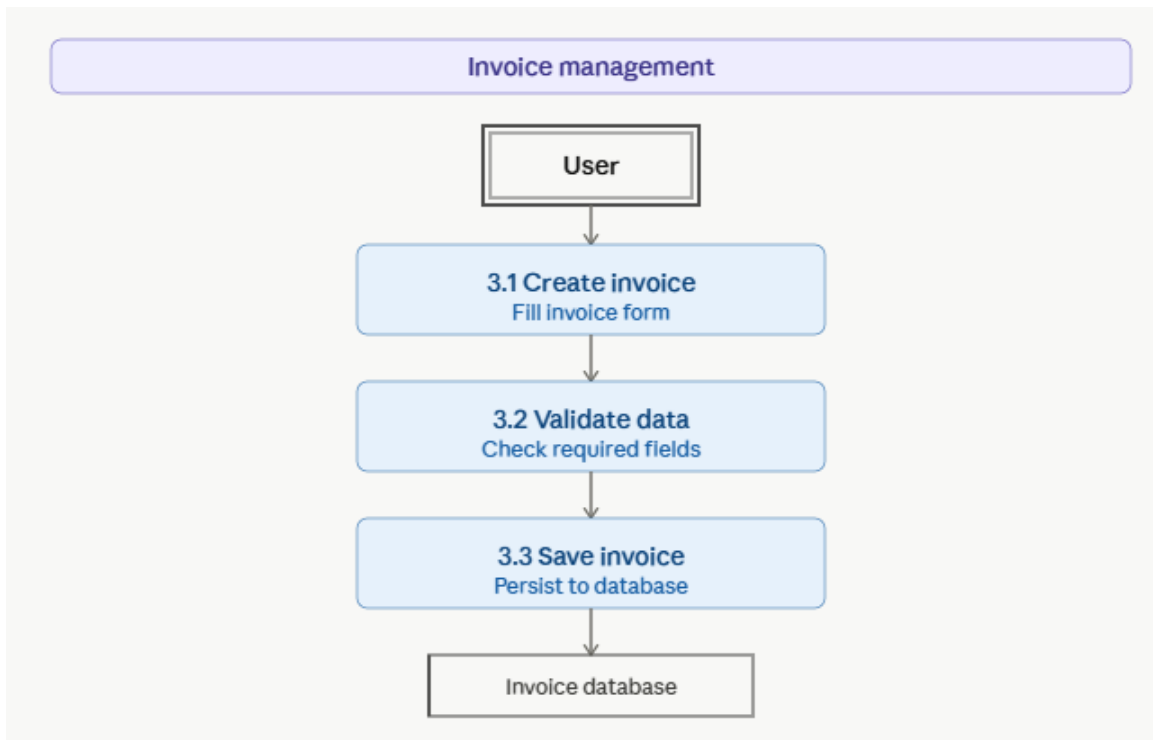
Breaks the system into major modules.

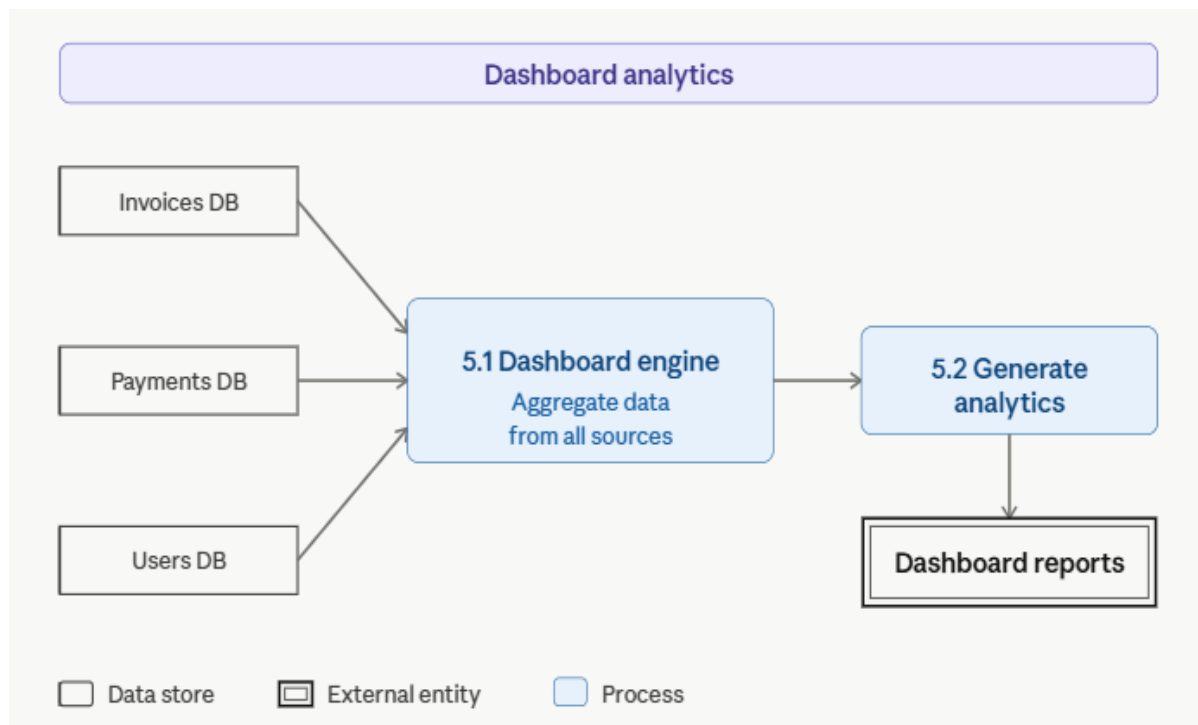


DFD LEVEL 2

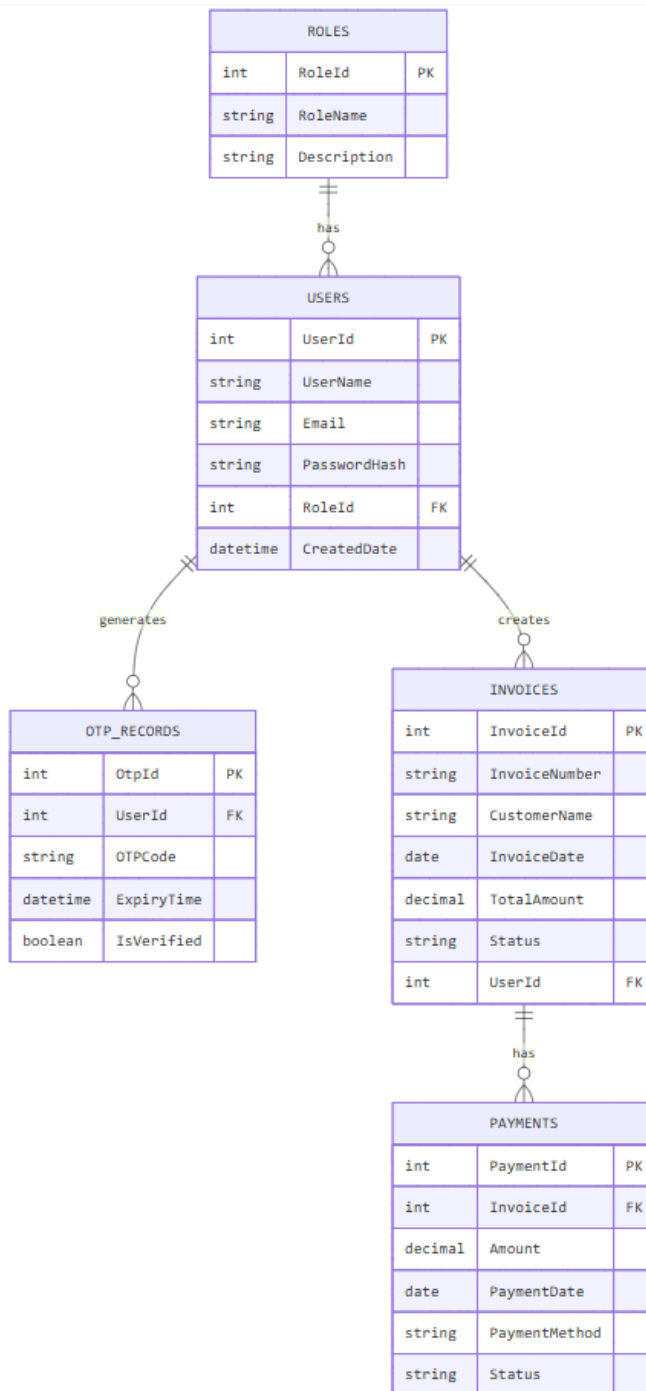
Detailed decomposition of Authentication, Invoice, Payment and Dashboard modules.







CHAPTER 8: ER DIAGRAM



CHAPTER 9: DATABASE DESIGN

9.1 Introduction

The Finance Billing & Analytics System uses Microsoft SQL Server as its primary database management system. The database is designed using relational database principles to ensure data consistency, integrity, security, and efficient retrieval of information.

The database consists of multiple interconnected tables that manage users, roles, invoices, payments, and OTP verification records. Primary keys and foreign keys are used to establish relationships between entities and maintain referential integrity.

9.2 User Table

Table Name: Users

Purpose

The Users table stores information about all registered users who can access the system.

Column Name	Data Type	Constraint	Description
UserId	INT	Primary Key	Unique User Identifier
UserName	NVARCHAR(100)	Unique	Username
Email	NVARCHAR(150)	Unique	User Email Address
PasswordHash	NVARCHAR(MAX)	Not Null	Encrypted Password
RoleId	INT	Foreign Key	Reference to Role Table
CreatedDate	DATETIME	Not Null	User Creation Date

Primary Key

- UserId

Foreign Key

- RoleId → Roles(RoleId)

Example

UserId	UserName	Email	RoleId
1	Admin	admin@gmail.com	1
2	FinanceUser	accountant@gmail.com	2

9.3 Role Table

Table Name: Roles

Purpose

The Roles table defines different user roles and permissions within the system.

Column Name	Data Type	Constraint	Description
RoleId	INT	Primary Key	Unique Role Identifier
RoleName	NVARCHAR(50)	Unique	Role Name
Description	NVARCHAR(255)	Nullable	Role Description

Primary Key

- RoleId

Example

RoleId	RoleName
1	Admin
2	Finance Manager
3	User

Relationship

One Role can be assigned to multiple Users.

Role (1) → Users (Many)

9.4 Invoice Table

Table Name: Invoices

Purpose

The Invoice table stores billing information generated for customers.

Column Name	Data Type	Constraint	Description
InvoiceId	INT	Primary Key	Unique Invoice Identifier
InvoiceNumber	NVARCHAR(50)	Unique	Invoice Number
CustomerName	NVARCHAR(150)	Not Null	Customer Name
InvoiceDate	DATETIME	Not Null	Invoice Creation Date
TotalAmount	DECIMAL(18,2)	Not Null	Invoice Amount
Status	NVARCHAR(50)	Not Null	Invoice Status
UserId	INT	Foreign Key	Created By User

Primary Key

- InvoiceId

Foreign Key

- UserId → Users(UserId)

Example

InvoiceId	InvoiceNumber	CustomerName	TotalAmount
101	INV001	ABC Ltd	15000
102	INV002	XYZ Pvt Ltd	25000

Relationship

One User can create multiple Invoices.

User (1) → Invoice (Many)

9.5 Payment Table

Table Name: Payments

Purpose

The Payments table stores all payment transactions associated with invoices.

Column Name	Data Type	Constraint	Description
PaymentId	INT	Primary Key	Unique Payment Identifier
InvoiceId	INT	Foreign Key	Invoice Reference
Amount	DECIMAL(18,2)	Not Null	Payment Amount
PaymentDate	DATETIME	Not Null	Date of Payment
PaymentMethod	NVARCHAR(50)	Not Null	Payment Mode
Status	NVARCHAR(50)	Not Null	Payment Status

Primary Key

- PaymentId

Foreign Key

- InvoiceId → Invoices(InvoiceId)

Example

PaymentId	InvoiceId	Amount
1	101	10000
2	101	5000

Relationship

One Invoice can have multiple Payments.

Invoice (1) → Payment (Many)

9.6 Database Relationships Summary

Parent Table	Child Table	Relationship
Roles	Users	One-to-Many
Users	Invoices	One-to-Many
Users	OTPRecords	One-to-Many
Invoices	Payments	One-to-Many

9.7 Database Design Advantages

- Maintains Referential Integrity
- Eliminates Data Redundancy
- Supports Secure Authentication
- Enables Efficient Invoice Tracking
- Facilitates Payment Management
- Supports Dashboard Analytics
- Scalable for Future Enhancements
- Optimized for Enterprise Applications

The database design follows normalization principles and industry-standard relational database practices, ensuring efficient storage, retrieval, and management of financial information.

CHAPTER 10: FRONTEND ARCHITECTURE

10.1 Introduction

The Frontend Architecture of the Finance Billing & Analytics System is developed using ASP.NET Core MVC, Razor Views, and Bootstrap. It provides a responsive and user-friendly interface through which users interact with the application.

The frontend is responsible for collecting user inputs, displaying dashboard analytics, presenting invoice and payment information, and communicating with the backend APIs.

10.2 Frontend Components

User Interface Layer

The User Interface consists of:

- Login Page
- Forgot Password Page
- OTP Verification Page
- Reset Password Page
- Dashboard
- Invoice Management Screens
- Payment Management Screens
- Role Management Screens

MVC Components

Model

The Model represents application data.

Examples:

- UserViewModel
- InvoiceViewModel
- PaymentViewModel
- LoginViewModel

View

Views are Razor (.cshtml) pages used to display data.

Examples:

- Login.cshtml
- Dashboard.cshtml
- InvoiceList.cshtml
- PaymentList.cshtml

Controller

Controllers handle user requests and communicate with APIs.

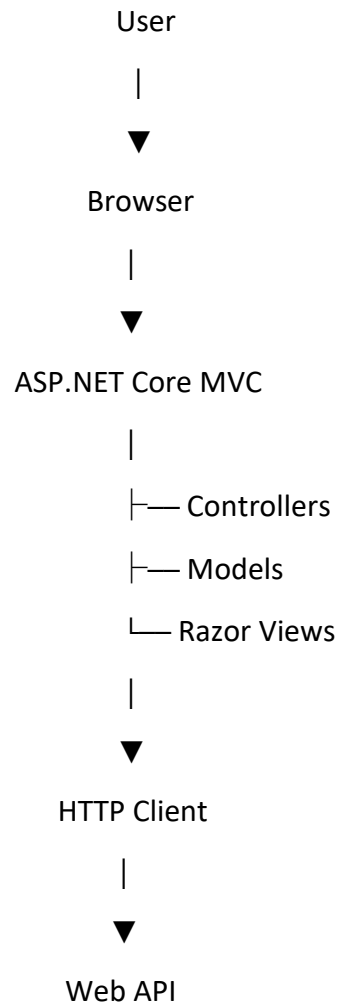
Examples:

- AuthController
- InvoiceController
- PaymentController
- DashboardController

10.3 Frontend Workflow

1. User accesses application.
2. MVC Controller receives request.
3. Controller calls Web API.
4. API returns JSON response.
5. Controller maps response to ViewModel.
6. Razor View displays data to user.

10.4 Frontend Architecture Diagram



The frontend layer focuses on presentation, validation, and user interaction while delegating business logic to the backend services.

CHAPTER 11: BACKEND ARCHITECTURE

11.1 Introduction

The Backend Architecture is developed using ASP.NET Core Web API and follows a layered architecture approach. It is responsible for processing requests, applying business rules, managing authentication, and interacting with the database.

11.2 Backend Layers

API Layer

The API Layer exposes RESTful endpoints that are consumed by the MVC application.

Modules:

- Authentication API
- Invoice API
- Payment API
- Dashboard API
- Role Management API

Business Logic Layer

Responsible for:

- Invoice Processing
- Payment Validation
- OTP Verification
- JWT Token Generation
- Authorization Rules
- Dashboard Calculations

Data Access Layer

Implemented using Entity Framework Core.

Responsibilities:

- CRUD Operations
- LINQ Queries
- Repository Pattern
- Database Transactions

Database Layer

Microsoft SQL Server stores all application data.

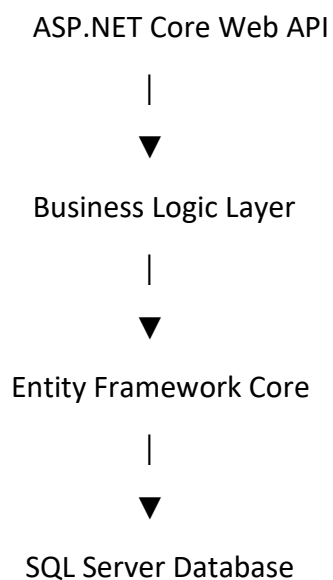
Tables:

- Users
- Roles
- Invoices
- Payments
- OTP Records

11.3 Backend Request Processing

1. API receives HTTP request.
2. JWT Token validation occurs.
3. Business logic is executed.
4. Entity Framework Core accesses database.
5. SQL Server returns data.
6. API sends JSON response.

11.4 Backend Architecture Diagram



CHAPTER 12: MVC ↔ API CONNECTIVITY FLOW

12.1 Introduction

The Finance Billing & Analytics System follows a distributed architecture where the ASP.NET Core MVC application acts as a client and consumes services exposed by the ASP.NET Core Web API.

The communication between MVC and API occurs through HTTP/HTTPS requests using HttpClient.

12.2 Connectivity Flow

Step 1: User Action

The user performs an action from the MVC application.

Examples:

- Login
- Create Invoice
- Add Payment
- View Dashboard

Step 2: MVC Controller Request

The MVC Controller receives the request and prepares an API call.

Example:

```
await _httpClient.PostAsJsonAsync();
```

Step 3: API Request

The request is sent to the Web API endpoint.

Example:

```
/api/Auth/login
```

```
/api/Invoice
```

```
/api/Payment
```

```
/api/Dashboard
```

Step 4: JWT Validation

The API validates the JWT token and authorizes the user.

Step 5: Business Logic Execution

The service layer processes business rules.

Examples:

- Validate User
- Verify OTP
- Calculate Dashboard Summary
- Validate Invoice Data

Step 6: Database Operations

Entity Framework Core performs CRUD operations on SQL Server.

Step 7: JSON Response

API returns response data in JSON format.

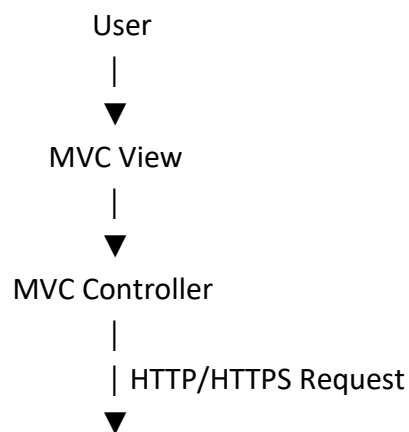
Example:

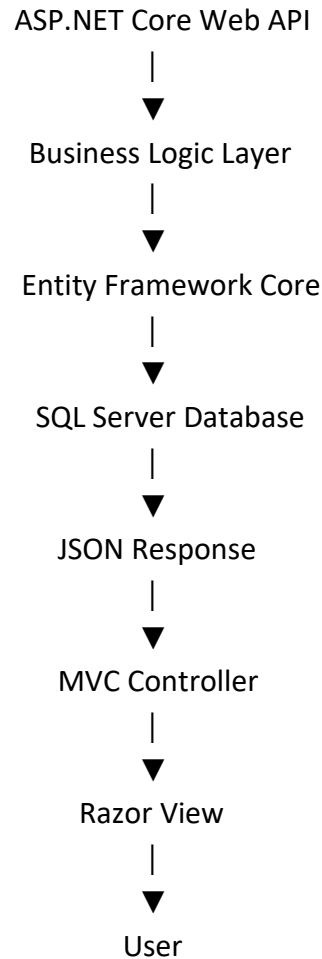
```
{  
  "success": true,  
  "message": "Invoice Created Successfully"  
}
```

Step 8: MVC View Rendering

The MVC application receives the response and updates the user interface.

12.3 MVC ↔ API Flow Diagram





12.4 Advantages of MVC ↔ API Architecture

- Better Separation of Concerns
- Reusable APIs
- Easier Maintenance
- Enhanced Security
- Scalable Architecture
- Supports Mobile and Third-Party Integration
- Independent Frontend and Backend Development

This architecture follows industry-standard enterprise application development practices and improves the overall maintainability and scalability of the system.

CHAPTER 13: AUTHENTICATION & AUTHORIZATION FLOW

13.1 Introduction

Authentication and Authorization are critical security mechanisms implemented in the Finance Billing & Analytics System. Authentication verifies the identity of users attempting to access the application, while Authorization determines the resources and functionalities a user can access based on assigned roles and permissions.

The system uses JWT (JSON Web Token) Authentication to secure API endpoints and Role-Based Authorization to restrict access to sensitive operations. This approach ensures that only authenticated and authorized users can perform specific actions within the application.

13.2 Authentication Flow

Authentication is the process of validating user credentials before granting access to the system.

Step 1: User Login

The user enters a valid username and password through the Login page.

Step 2: Request Submission

The MVC application sends the login request to the Authentication API.

Step 3: Credential Validation

The API validates the provided credentials against records stored in the Users table.

Step 4: Password Verification

The entered password is compared with the stored encrypted password using BCrypt hashing.

Step 5: JWT Token Generation

Upon successful validation, the system generates a JWT token containing user identity and role information.

Step 6: Access Granted

The token is returned to the MVC application and stored securely for future requests.

13.3 Authorization Flow

Authorization determines the actions that authenticated users are permitted to perform.

Step 1: User Sends Request

The user accesses a secured page or API endpoint.

Step 2: Token Validation

The API validates the JWT token attached to the request.

Step 3: Role Extraction

User role information is extracted from token claims.

Step 4: Permission Verification

The system checks whether the user has permission to access the requested resource.

Step 5: Access Decision

The request is either approved or denied based on authorization policies.

13.4 Authentication and Authorization Benefits

The implementation provides:

- Enhanced Security
- Role-Based Access Control
- Secure API Communication
- Protection Against Unauthorized Access
- Scalable Security Architecture

CHAPTER 14: JWT IMPLEMENTATION

14.1 Introduction

JSON Web Token (JWT) is used to implement secure token-based authentication within the Finance Billing & Analytics System. JWT enables stateless authentication by securely transmitting user identity information between the client and server.

14.2 JWT Implementation Process

Step 1: User Authentication

The user submits login credentials through the Login page.

Step 2: Credential Validation

The Authentication API validates the credentials using the Users table.

Step 3: Token Creation

The system generates a JWT token containing:

- User ID
- Username
- User Role
- Token Expiration Time

Step 4: Token Delivery

The generated token is returned to the MVC application.

Step 5: Token Storage

The token is securely stored in the user session.

Step 6: API Requests

The token is attached to all subsequent API requests.

Step 7: Token Validation

The API validates the token before processing any request.

Step 8: Authorized Access

If validation succeeds, the requested operation is executed.

14.3 Advantages of JWT

- Stateless Authentication
- Improved Security
- Better Scalability
- Faster Authentication Process
- Reduced Server Load

CHAPTER 15: FORGOT PASSWORD AND OTP FLOW

15.1 Introduction

The Forgot Password module allows users to securely recover access to their accounts when they forget their passwords. The process uses OTP (One-Time Password) verification to ensure account security.

15.2 OTP Verification Process

Step 1: Forgot Password Request

The user selects the Forgot Password option.

Step 2: Email Submission

The registered email address is provided.

Step 3: OTP Generation

The system generates a unique OTP.

Step 4: OTP Storage

The generated OTP is stored in the database along with expiration details.

Step 5: Email Delivery

The OTP is sent to the registered email address.

Step 6: OTP Verification

The user enters the received OTP.

Step 7: Validation

The system verifies:

- OTP Accuracy
- OTP Expiration Time
- Associated User Information

Step 8: Password Reset

Upon successful verification, the user is allowed to create a new password.

Step 9: Password Encryption

The new password is encrypted using BCrypt hashing before storage.

15.3 Benefits of OTP Verification

- Enhanced Account Security
- Prevention of Unauthorized Access
- Secure Password Recovery
- Improved User Experience

CHAPTER 16: INVOICE MANAGEMENT MODULE

16.1 Introduction

The Invoice Management Module is one of the core components of the Finance Billing & Analytics System. It enables users to create, update, view, and manage invoices efficiently while maintaining accurate financial records.

16.2 Module Features

The Invoice Management Module provides the following functionalities:

- Create Invoice
- View Invoice
- Update Invoice
- Delete Invoice
- Invoice Status Tracking
- Invoice Reporting

16.3 Invoice Creation Process

Step 1: Invoice Entry

The user enters invoice details.

Step 2: Data Validation

The system validates:

- Invoice Number
- Customer Information
- Amount
- Invoice Date

Step 3: Business Rule Validation

Business rules are applied before processing.

Step 4: Database Storage

The validated invoice is stored in the Invoice table.

Step 5: Confirmation

A success response is returned to the user.

16.4 Business Rules

The following business rules are implemented:

- Invoice Number must be unique.
- Invoice Amount must be greater than zero.
- Customer Name is mandatory.
- Invoice Date cannot be empty.
- Only authorized users can create invoices.
- JWT authentication is required for all invoice operations.

16.5 Module Benefits

- Improved Invoice Tracking
- Reduced Manual Errors
- Centralized Billing Information
- Faster Processing
- Enhanced Financial Management

CHAPTER 17: PAYMENT MANAGEMENT MODULE

17.1 Introduction

The Payment Management Module is responsible for recording, monitoring, and managing payment transactions associated with invoices. This module ensures that all payments are accurately tracked and linked to their corresponding invoices, thereby maintaining financial transparency and accountability.

The module enables users to add payment details, view payment history, update payment records, and monitor payment status. It also supports financial reporting and dashboard analytics by providing real-time payment information.

17.2 Module Features

The Payment Management Module provides the following functionalities:

- Add Payment
- View Payment Details
- Update Payment Information
- Delete Payment Records
- Payment Tracking
- Payment Status Monitoring
- Financial Reporting Support

17.3 Payment Processing Workflow

Step 1: Payment Entry

The user enters payment details through the Payment Management interface.

Step 2: Invoice Verification

The system verifies that the selected invoice exists in the database.

Step 3: Data Validation

The entered payment information is validated before processing.

Step 4: Database Storage

Validated payment information is stored in the Payments table.

Step 5: Confirmation

The system returns a successful response indicating that the payment has been recorded.

17.4 Business Rules

The following business rules are implemented:

- Payment must be linked to an existing invoice.
- Payment amount must be greater than zero.
- Payment date cannot be empty.
- Authorized users only can perform payment operations.
- JWT authentication is mandatory for payment APIs.

17.5 Benefits of Payment Management

- Accurate Payment Tracking
- Improved Financial Transparency
- Reduced Data Entry Errors
- Better Invoice Reconciliation
- Enhanced Financial Reporting

CHAPTER 18: DASHBOARD ANALYTICS MODULE

18.1 Introduction

The Dashboard Analytics Module provides real-time financial insights and summary reports to help users monitor organizational performance. The dashboard aggregates information from invoices, payments, and other financial records and presents it in a visually understandable format.

This module enables users to make informed decisions by providing key performance indicators and financial statistics.

18.2 Dashboard Features

The dashboard includes:

- Total Invoices
- Total Payments
- Revenue Summary
- Collection Rate Analysis
- Invoice Summary
- Payment Summary
- Financial Overview

18.3 Dashboard Workflow

Step 1: Data Retrieval

The dashboard requests data from the Dashboard API.

Step 2: Data Aggregation

The API retrieves invoice and payment information from SQL Server.

Step 3: Business Calculations

Financial calculations and summaries are generated.

Step 4: Response Generation

The API returns summarized data in JSON format.

Step 5: Dashboard Display

The MVC application displays analytical information through the dashboard interface.

18.4 Benefits of Dashboard Analytics

- Real-Time Monitoring
- Improved Decision-Making
- Better Financial Visibility
- Performance Tracking
- Enhanced Reporting Capabilities

CHAPTER 19: SWAGGER API DOCUMENTATION

19.1 Introduction

Swagger is integrated into the Finance Billing & Analytics System to provide interactive API documentation and testing capabilities. Swagger simplifies API development by offering a user-friendly interface for viewing and testing available endpoints.

The system uses Swagger/OpenAPI to document all Web API endpoints.

19.2 Purpose of Swagger

Swagger helps developers and testers:

- Understand API functionality
- Test API endpoints
- Verify request and response formats
- Validate authentication mechanisms
- Troubleshoot API issues

19.3 API Modules Documented in Swagger

Authentication APIs

- Register
- Login
- Forgot Password
- Verify OTP
- Reset Password

Invoice APIs

- Create Invoice
- View Invoices
- Update Invoice

- Delete Invoice

Payment APIs

- Create Payment
- View Payments
- Update Payment
- Delete Payment

Dashboard APIs

- Dashboard Summary

Role APIs

- Create Role
- View Roles
- Delete Role

19.4 Swagger Benefits

- Interactive API Testing
- Automatic Documentation
- Faster Development
- Improved API Understanding
- Easier Debugging

19.5 JWT Integration with Swagger

The project integrates JWT authentication within Swagger, allowing authorized API testing through the Swagger interface.

This ensures that protected endpoints can be tested securely.

CHAPTER 20: UNIT TESTING

20.1 Introduction

Unit Testing is performed to verify that individual components of the application function correctly. It helps identify defects early in the development lifecycle and improves software reliability.

The Finance Billing & Analytics System follows a testing strategy that validates critical functionalities including authentication, invoice processing, payment management, and business logic execution.

20.2 Objectives of Unit Testing

The objectives of unit testing include:

- Validate business logic
- Verify API functionality
- Detect errors early
- Improve code quality
- Increase application reliability

20.3 Components Tested

The following components are tested:

Authentication Module

Tests include:

- Login Validation
- JWT Token Generation
- Authorization Checks
- OTP Verification

Invoice Module

Tests include:

- Invoice Creation
- Invoice Update
- Invoice Retrieval
- Invoice Deletion

Payment Module

Tests include:

- Payment Creation
- Payment Validation
- Payment Retrieval
- Payment Deletion

Dashboard Module

Tests include:

- Dashboard Summary Generation
- Revenue Calculation
- Collection Analysis

20.4 Testing Process

Step 1: Test Case Creation

Test cases are prepared for each module.

Step 2: Execution

Individual methods and APIs are executed with test data.

Step 3: Validation

Actual results are compared against expected outcomes.

Step 4: Defect Resolution

Any identified defects are corrected and retested.

CHAPTER 21: SECURITY BEST PRACTICES

21.1 Introduction

Security is a critical aspect of the Finance Billing & Analytics System because it handles sensitive user information, financial records, invoices, and payment data. The application follows industry-standard security practices to ensure confidentiality, integrity, and availability of data.

21.2 Security Measures Implemented

JWT Authentication

The application uses JSON Web Tokens (JWT) to secure API communication and verify user identity.

Benefits:

- Stateless Authentication
- Secure Access Control
- Improved Scalability
- Reduced Server Load

Password Encryption

User passwords are encrypted using BCrypt hashing before being stored in the database.

Benefits:

- Secure Password Storage
- Protection Against Brute Force Attacks
- Enhanced User Security

Role-Based Authorization

Access to system resources is controlled through role-based authorization.

Roles include:

- Administrator

- Finance Manager
- User

Benefits:

- Controlled Access
- Improved Data Security
- Reduced Unauthorized Activities

Input Validation

All user inputs are validated before processing.

Benefits:

- Prevention of Invalid Data Entry
- Protection Against Injection Attacks
- Improved Data Integrity

Secure API Access

All protected API endpoints require authentication tokens.

Benefits:

- Restricted API Access
- Enhanced Security
- Better Access Management

21.3 Security Benefits

- Data Confidentiality
- Secure User Authentication
- Controlled Access Permissions
- Improved System Reliability
- Protection Against Common Security Threats

CHAPTER 22: BUSINESS RULES

22.1 Introduction

Business rules define the operational guidelines that govern how the Finance Billing & Analytics System processes data and performs transactions.

These rules ensure consistency, accuracy, and compliance with organizational requirements.

22.2 User Management Rules

- Username must be unique.
- Email address must be unique.
- Passwords must be encrypted before storage.
- Users must belong to a valid role.
- Only authenticated users can access the system.

22.3 Authentication Rules

- JWT token is required for secured APIs.
- Invalid tokens are rejected.
- Expired tokens cannot access protected resources.
- OTP verification is mandatory during password reset.

22.4 Invoice Rules

- Invoice Number must be unique.
- Invoice Date cannot be empty.
- Invoice Amount must be greater than zero.
- Customer information is mandatory.
- Authorized users only can create invoices.

22.5 Payment Rules

- Payment must be linked to an existing invoice.
- Payment Amount must be greater than zero.
- Payment Date is mandatory.
- Unauthorized users cannot perform payment operations.

22.6 Dashboard Rules

- Dashboard data must be generated from valid invoice and payment records.
- Financial calculations must use current database information.
- Dashboard summaries must be updated dynamically.

CHAPTER 23: ASSUMPTIONS AND IDEAL SCENARIOS

23.1 Assumptions

The following assumptions were made during project development:

- Users have valid internet connectivity.
- SQL Server database is available and operational.
- Registered email addresses are valid.
- SMTP service is configured correctly for OTP delivery.
- Users possess appropriate system permissions.
- API services are available during system operation.

23.2 Ideal System Scenario

Under ideal conditions:

1. User logs into the system successfully.
2. JWT token is generated.
3. User accesses authorized modules.
4. Invoice information is managed accurately.
5. Payments are recorded correctly.
6. Dashboard displays real-time analytics.
7. OTP emails are delivered instantly.
8. Database operations complete without errors.

23.3 System Constraints

- System depends on database availability.
- Email services are required for OTP functionality.
- Network connectivity is necessary for API communication.

CHAPTER 24: COMPARISON WITH REAL BILLING APPLICATIONS

24.1 Introduction

The Finance Billing & Analytics System incorporates many features commonly found in enterprise billing and financial management applications.

Examples of similar systems include:

- SAP ERP
- Oracle Financials
- Zoho Books
- QuickBooks
- Tally Prime

24.2 Feature Comparison

Feature	Project System	Real Billing Applications
User Authentication	✓	✓
JWT Security	✓	✓
Role Management	✓	✓
Invoice Management	✓	✓
Payment Tracking	✓	✓
Dashboard Analytics	✓	✓
OTP Verification	✓	✓
API Documentation	✓	✓
Cloud Deployment	Future Scope	✓

Payment Gateway	Future Scope	✓
GST Compliance	Future Scope	✓

24.3 Similarities

- Secure Authentication
- Invoice Processing
- Payment Management
- Dashboard Reporting
- User Role Management
- Financial Data Tracking

24.4 Differences

Real enterprise systems typically provide:

- Multi-Branch Management
- GST Compliance
- Payment Gateway Integration
- Advanced Reporting
- Audit Logs
- Cloud Infrastructure
- AI-Based Forecasting

These features can be added in future versions of the project.

CHAPTER 25: FUTURE ENHANCEMENTS

25.1 Introduction

The current implementation provides a strong foundation for financial management. However, several advanced features can be incorporated in future versions to improve functionality and scalability.

25.2 Proposed Enhancements

GST Management Module

- GST Calculation
- GST Reports
- Tax Compliance Features

Payment Gateway Integration

- Razorpay Integration
- Stripe Integration
- Online Payment Processing

Email Notifications

- Invoice Notifications
- Payment Reminders
- Account Activity Alerts

SMS-Based OTP Verification

- Mobile Verification
- Multi-Factor Authentication

Advanced Analytics

- Interactive Charts
- Revenue Forecasting
- Financial Trends Analysis

Audit Trail System

- User Activity Tracking
- Change History Monitoring

Mobile Application

- Android Application
- iOS Application

Cloud Deployment

- Microsoft Azure
- Amazon Web Services (AWS)

25.3 Benefits of Future Enhancements

- Increased Scalability
- Better User Experience
- Enhanced Security
- Advanced Reporting Capabilities
- Improved Business Intelligence

CHAPTER 26: CONCLUSION

26.1 Conclusion

The Finance Billing & Analytics System successfully achieves its objective of providing a secure, efficient, and user-friendly platform for managing financial operations. The system integrates invoice management, payment tracking, dashboard analytics, authentication, authorization, and OTP-based password recovery into a single web application.

The application is developed using modern technologies including ASP.NET Core MVC, ASP.NET Core Web API, Entity Framework Core, SQL Server, JWT Authentication, and Swagger API Documentation. The layered architecture ensures scalability, maintainability, and security while following industry-standard software development practices.

The implemented modules effectively automate financial processes, reduce manual effort, improve data accuracy, and provide real-time insights through analytical dashboards. Security features such as BCrypt password hashing, JWT authentication, and role-based authorization help protect sensitive financial information.

Overall, the Finance Billing & Analytics System serves as a reliable financial management solution capable of supporting organizational billing operations while providing a strong foundation for future enhancements such as GST integration, payment gateways, cloud deployment, and advanced analytics.

26.2 Project Outcome

The project successfully demonstrates:

- Secure User Authentication and Authorization
- Invoice Management
- Payment Tracking
- Dashboard Analytics
- OTP-Based Password Recovery
- RESTful API Development
- Database Management

CHAPTER 26: SCREEN SHOT

26.1 FRONTEND:

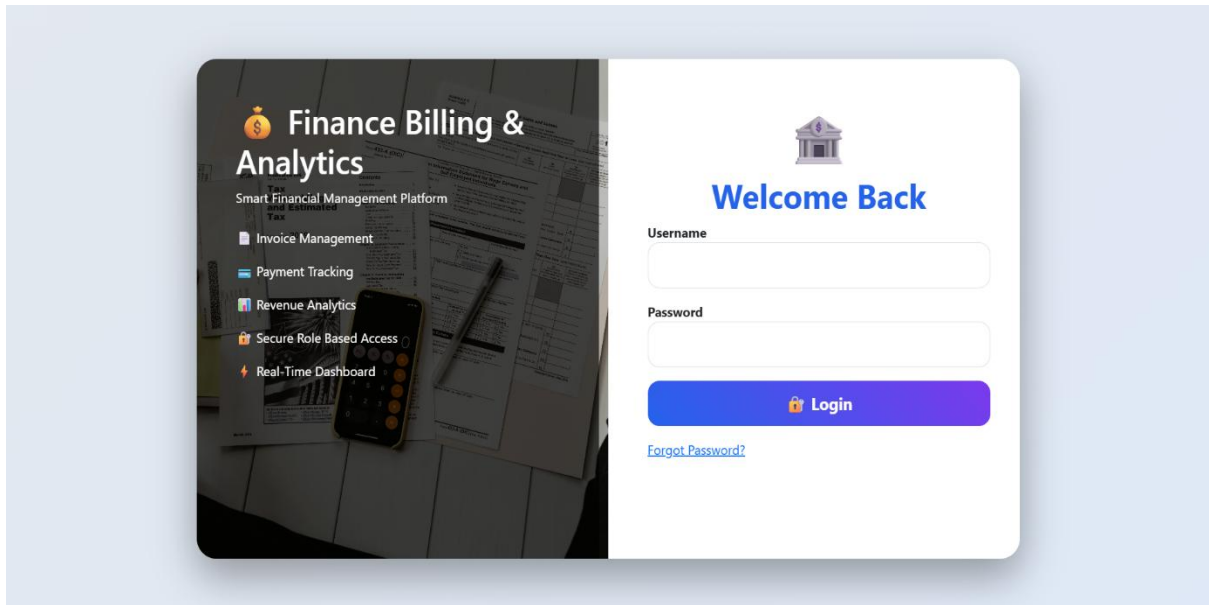


Fig.1



Fig.2

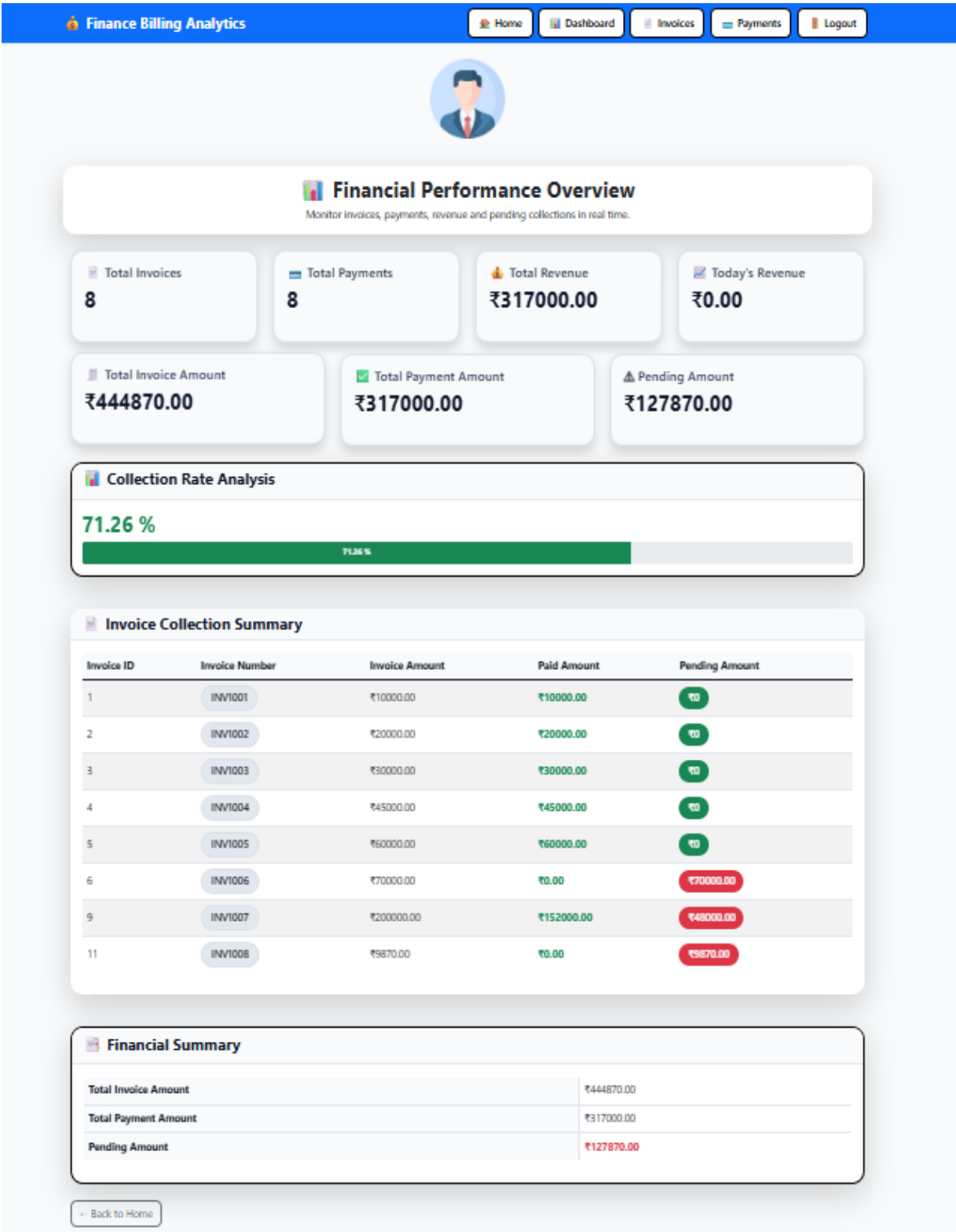


Fig.3

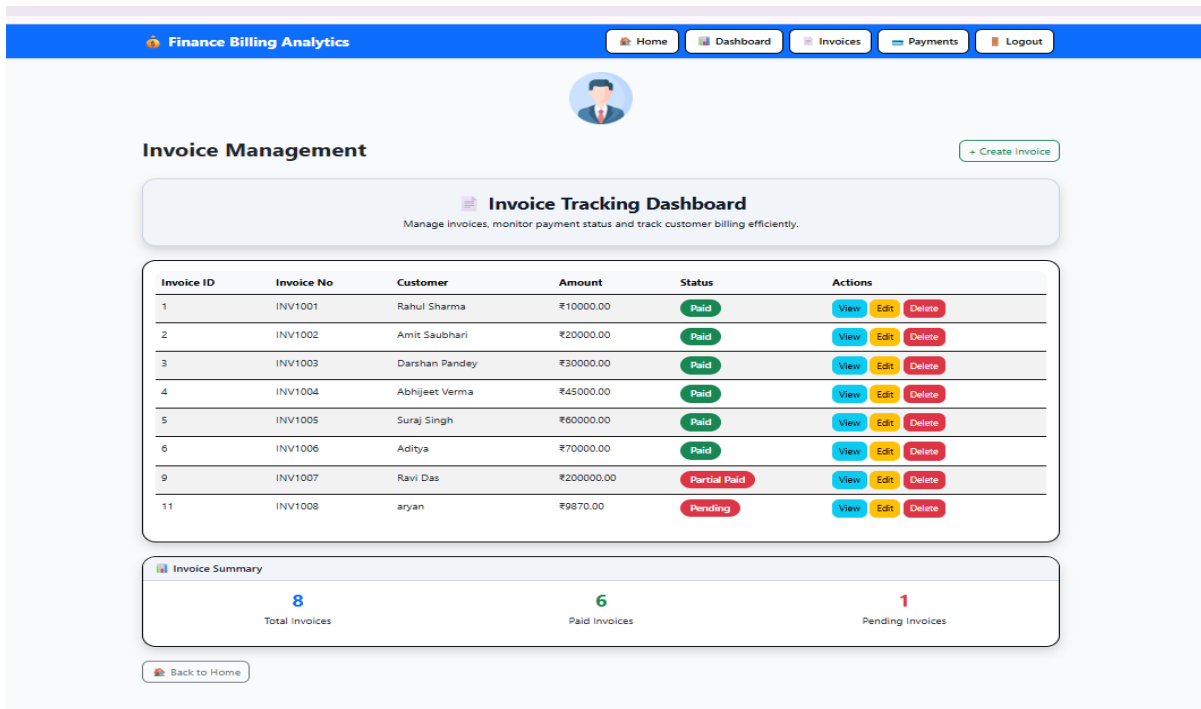


Fig.4

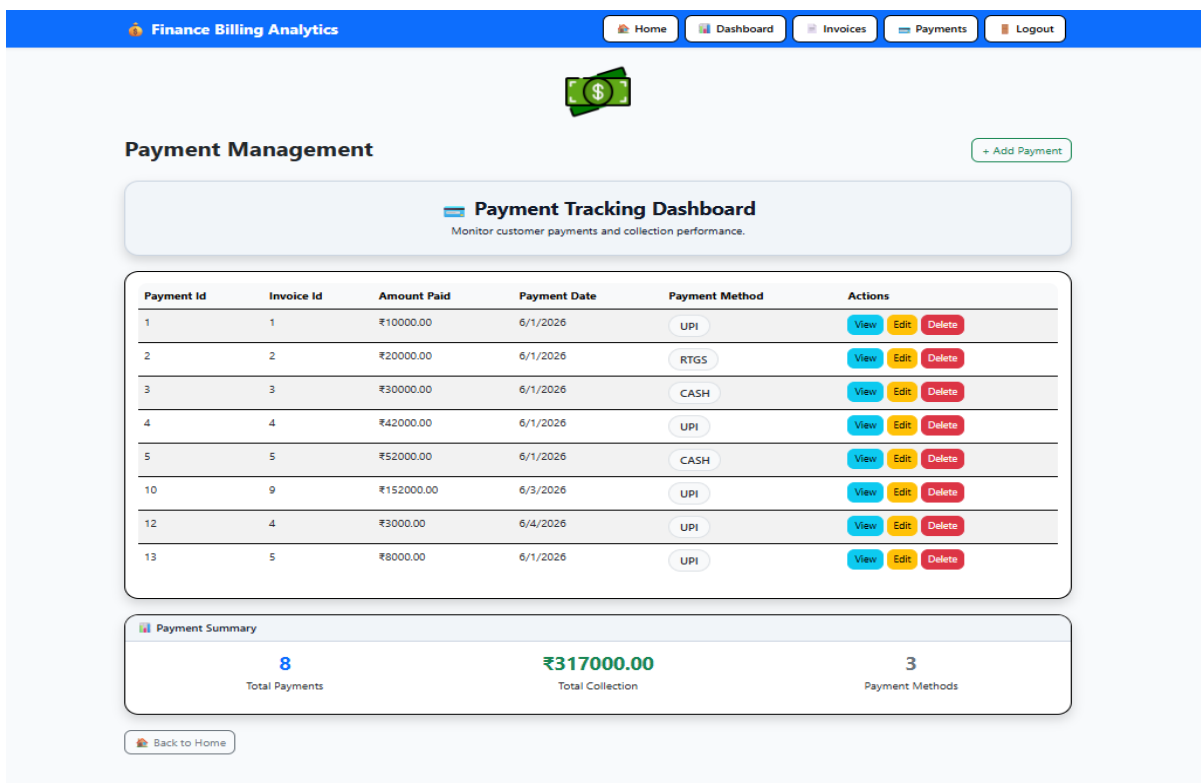


Fig.5

26.2 BACKEND:

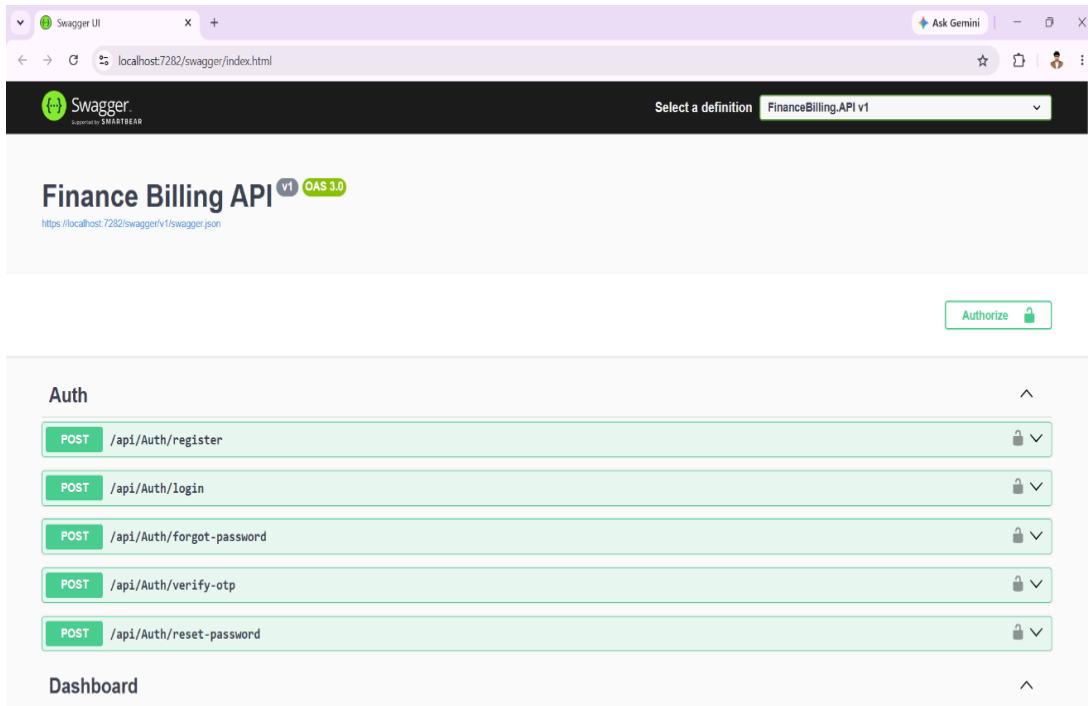


Fig.6

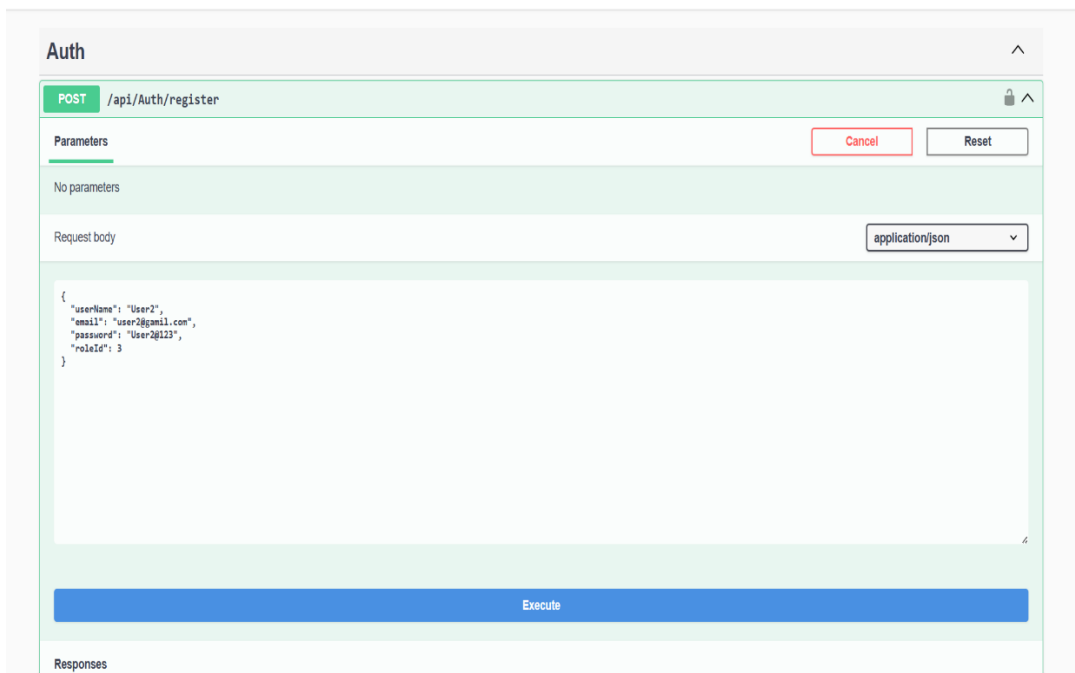


Fig.7

Responses

Curl

```
curl -X 'POST' \
  'https://localhost:7282/api/Auth/register' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "userName": "User2",
    "email": "user2@gmail.com",
    "password": "User2@123",
    "roleId": 3
  }'
```

Request URL

https://localhost:7282/api/Auth/register

Server response

Code	Details
200	Response body User Registered Response headers content-type: text/plain; charset=utf-8 date: Fri, 05 Jun 2026 04:17:30 GMT server: Kestrel

Responses

Code	Description	Links
200	OK	No links

Fig.8

POST /api/Auth/login

Parameters

No parameters

Request body

application/json

```
{
  "userName": "User2",
  "password": "User2@123"
}
```

Execute Clear

Responses

Fig.9

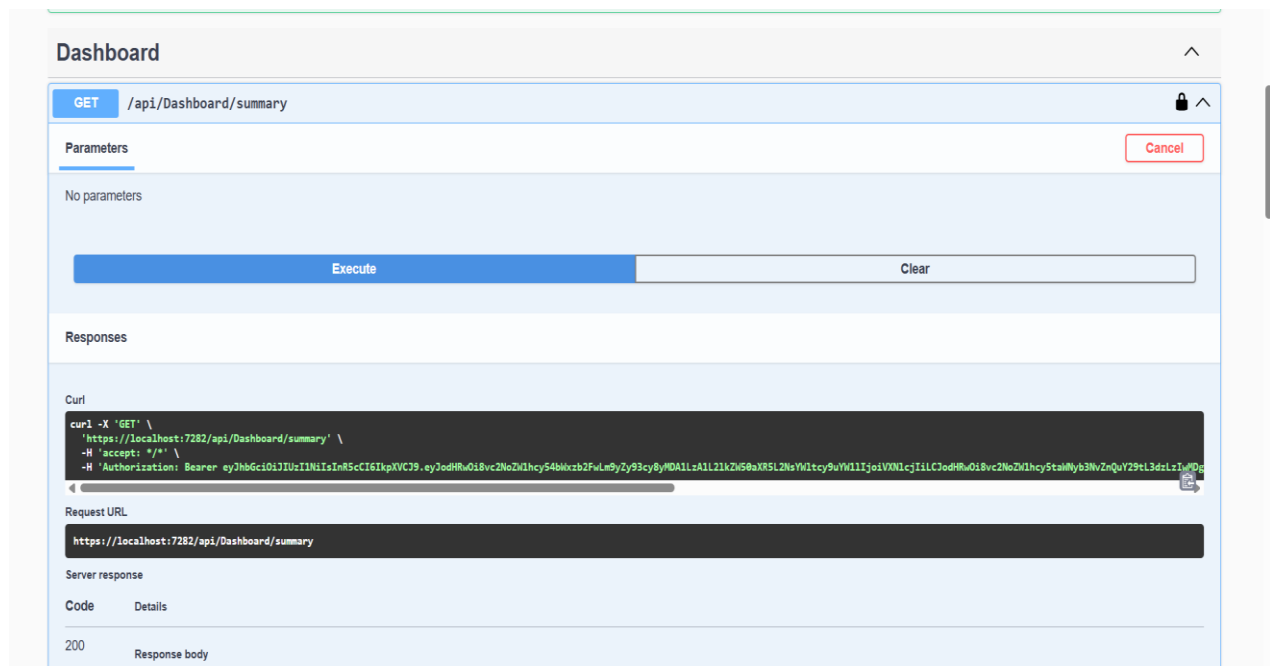


Fig.12

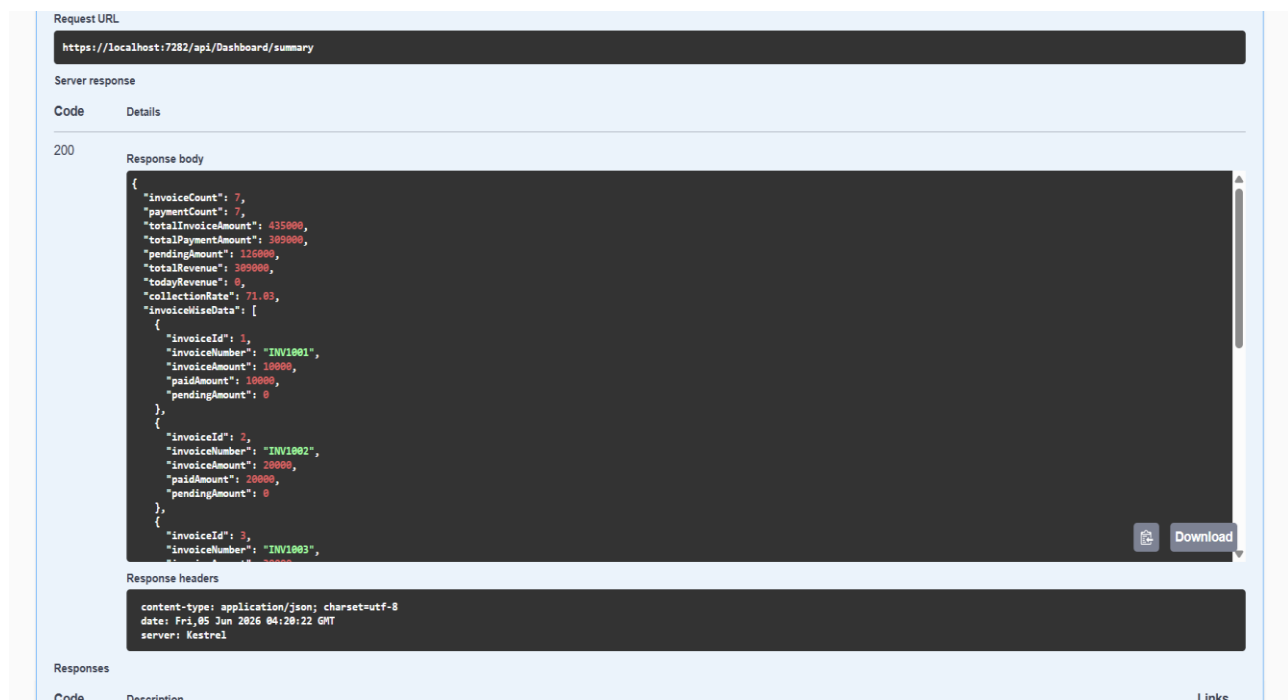


Fig.13

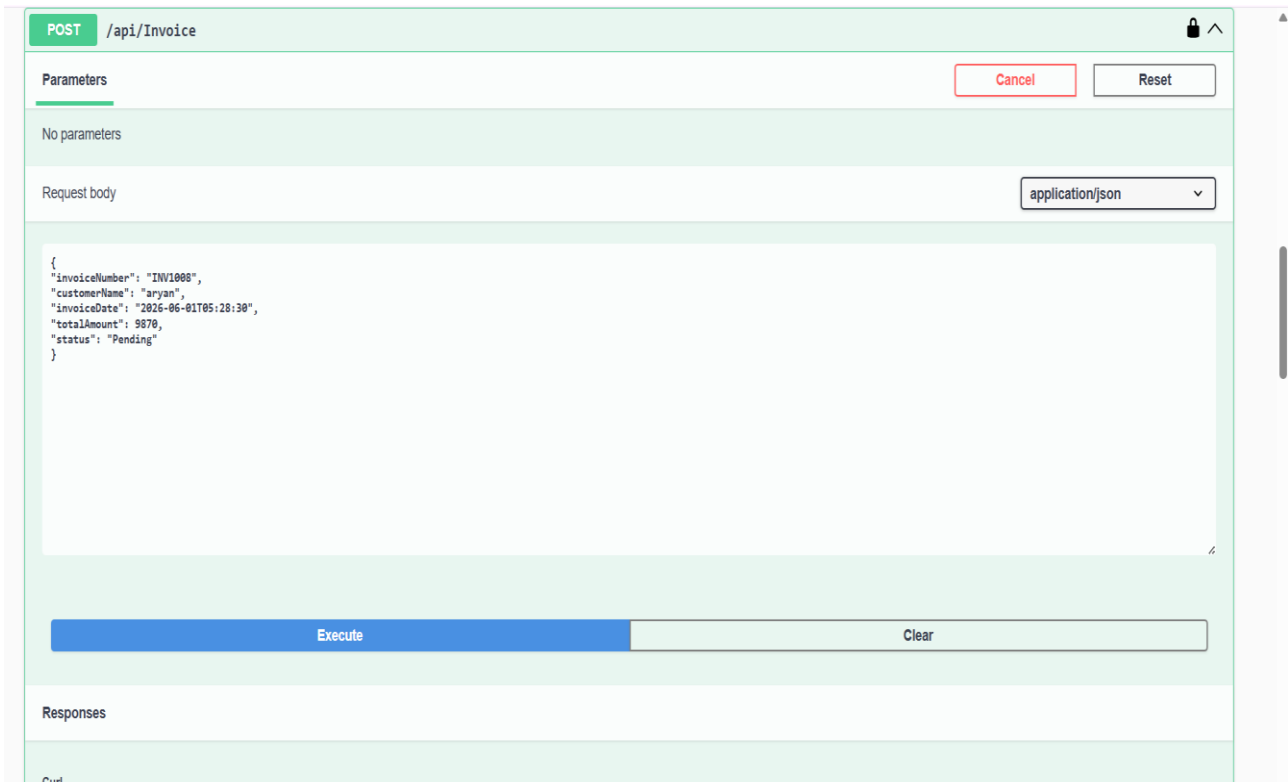
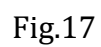


Fig.14



Fig.15



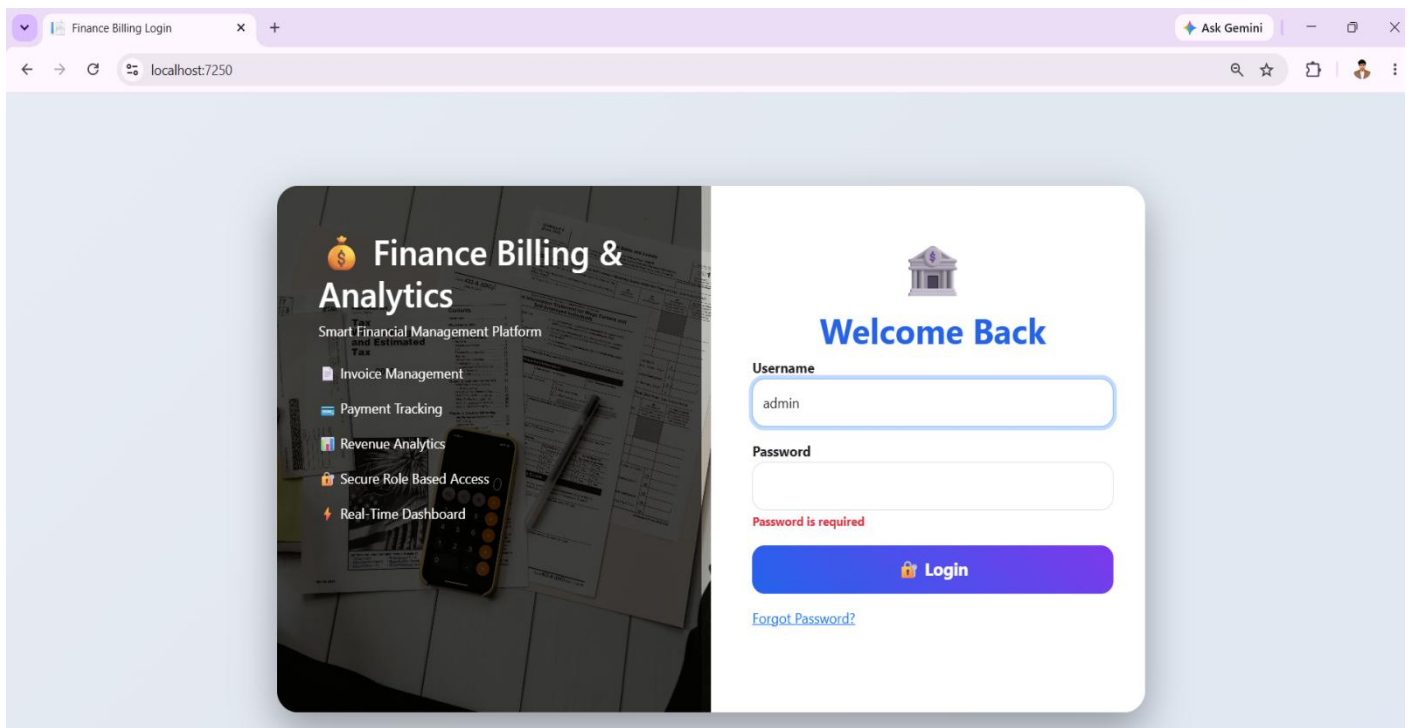


Fig.22

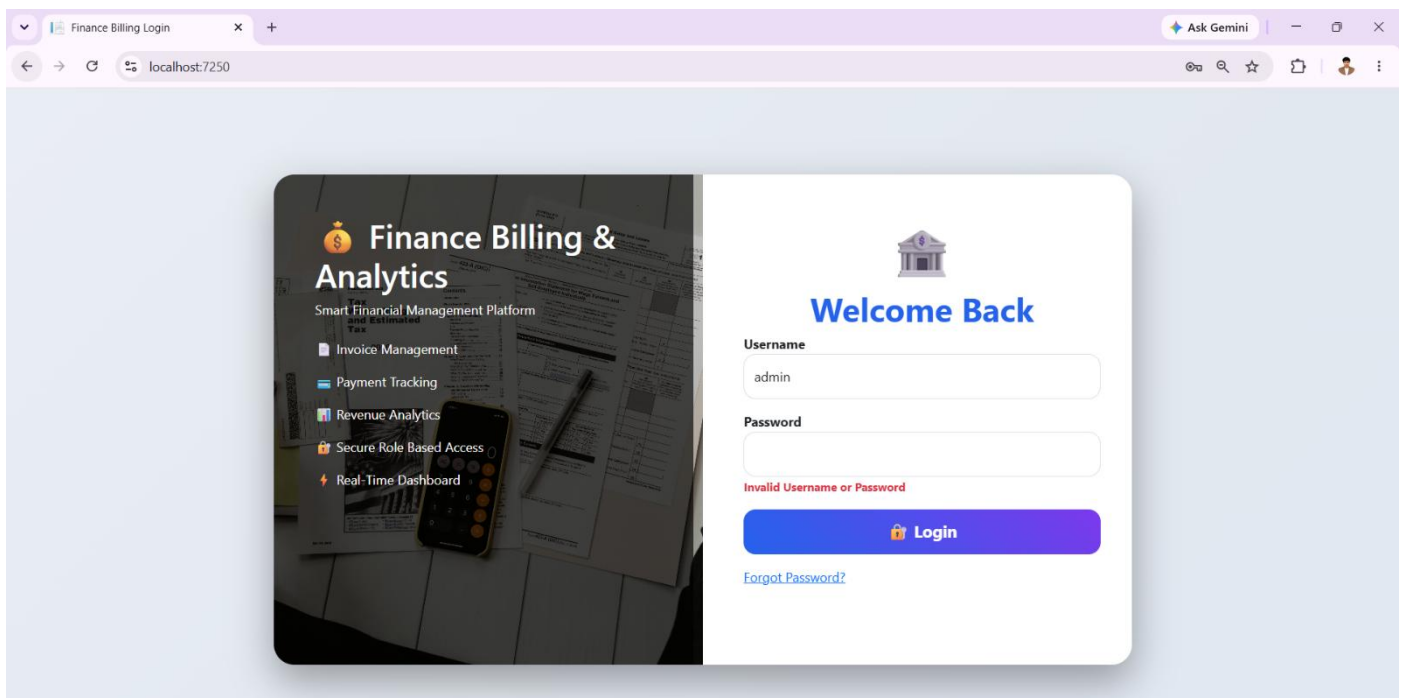


Fig.23

← → ↻ localhost:7250/Invoice/Create 🔍 ☆ 📄 👤 ⋮

Finance Billing Analytics Home Dashboard Invoices Payments Logout

Create New Invoice

InvoiceNumber

Invoice Number is required

CustomerName

Customer Name is required

TotalAmount

Amount is required

Status
-- Select Status --
The Status field is required.

Save Invoice ← Back to Invoices

Fig.24

Add New Payment

InvoiceId

Invoice Id is required

AmountPaid

Amount Paid is required

PaymentDate
 📅
Payment Date is required

PaymentMethod
-- Select Payment Method --
Payment Method is required

Save Payment ← Back to Payments

Fig.25

The screenshot shows a web browser window with the address bar displaying 'localhost:7250/Auth/ForgotPassword'. The page title is 'Finance Billing Analytics'. The main content area features a light gray box with the heading 'Forgot Password'. Below the heading is a text input field labeled 'Email Address'. A red error message 'Email is required' is displayed below the input field. At the bottom of the box are two buttons: a blue 'Send OTP' button and a gray 'Back' button.

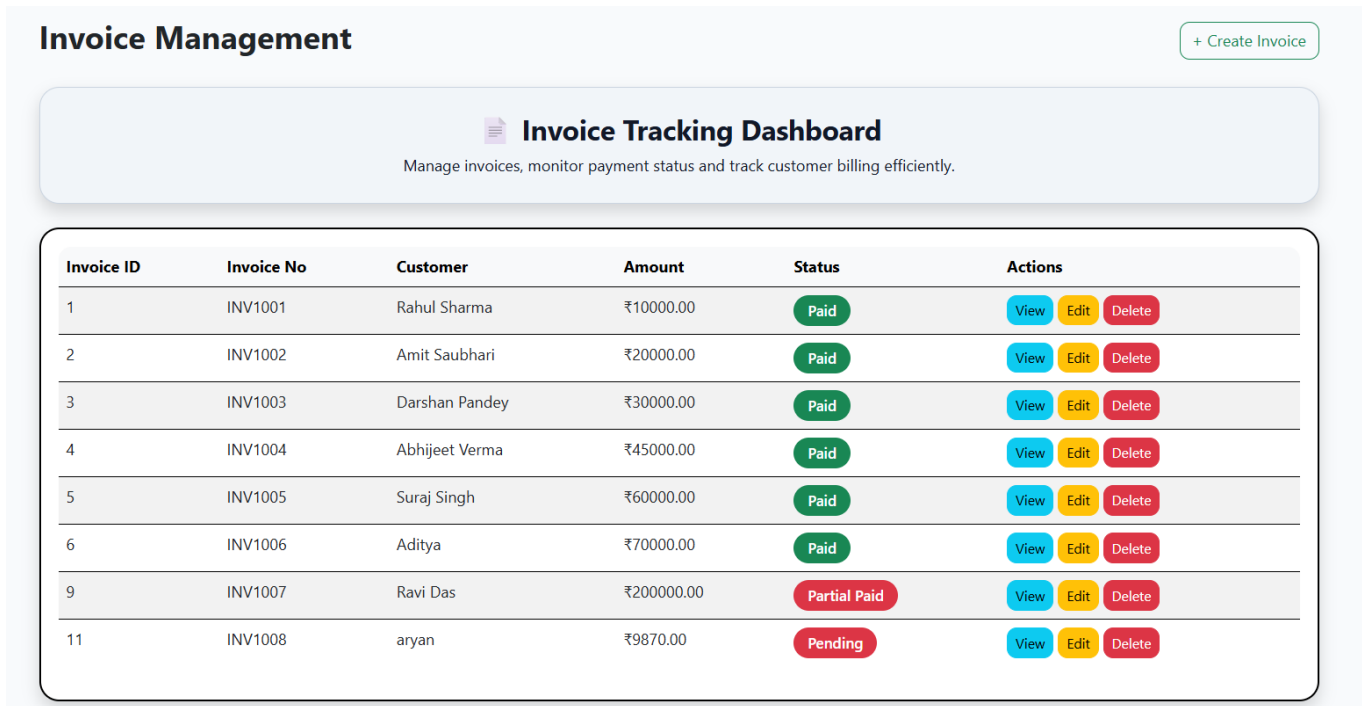
Fig.26

The screenshot shows a web browser window with the address bar displaying 'localhost:7250/Auth/VerifyOtp'. The page title is 'Finance Billing Analytics'. The main content area features a light gray box with the heading 'Verify OTP'. Below the heading are two text input fields: 'Email' (containing 'admin@gmail.com') and 'OTP'. A red error message 'OTP is required' is displayed below the OTP input field. At the bottom of the box are two buttons: a green 'Verify OTP' button and a gray 'Back' button.

Fig.27

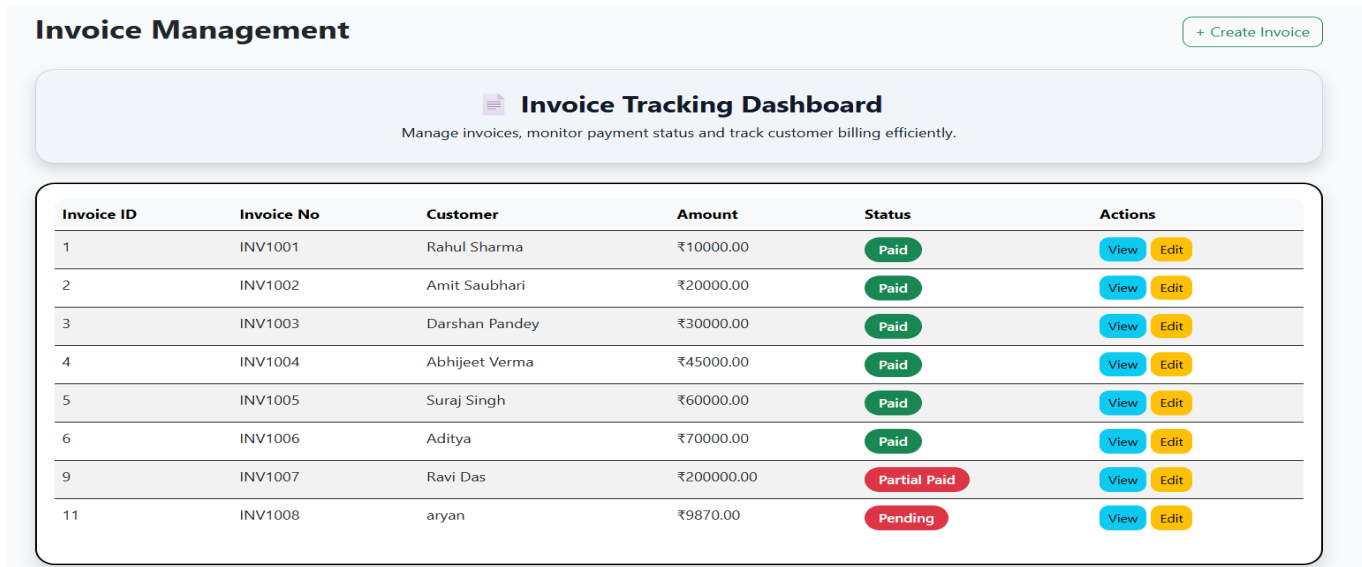
The screenshot shows a web browser window with the address bar displaying 'localhost:7250/Auth/ResetPassword'. The page title is 'Finance Billing Analytics'. The main content area features a light gray box with the heading 'Reset Password'. Below the heading is a text input field labeled 'New Password'. A red error message 'Password is required' is displayed below the input field. At the bottom of the box are two buttons: a red 'Reset Password' button and a gray 'Back' button.

Fig.28



(When Admin Login then It can Create,View,detail,Edit All Invoice and Payment)

Fig.29



(When Accountant/Finance Manager Login then It can Create ,View, Edit but not able to delete Invoice and Payment)

Fig.30

Invoice Management

Invoice Tracking Dashboard

Manage invoices, monitor payment status and track customer billing efficiently.

Invoice ID	Invoice No	Customer	Amount	Status	Actions
1	INV1001	Rahul Sharma	₹10000.00	Paid	<button>View</button>
2	INV1002	Amit Saubhari	₹20000.00	Paid	<button>View</button>
3	INV1003	Darshan Pandey	₹30000.00	Paid	<button>View</button>
4	INV1004	Abhijeet Verma	₹45000.00	Paid	<button>View</button>
5	INV1005	Suraj Singh	₹60000.00	Paid	<button>View</button>
6	INV1006	Aditya	₹70000.00	Paid	<button>View</button>
9	INV1007	Ravi Das	₹200000.00	Partial Paid	<button>View</button>
11	INV1008	aryan	₹9870.00	Pending	<button>View</button>

(When Viewer/User can Login then It can only View Invoice and Payment)

Fig.30

26.4 INVOICE MANAGEMENT MODULE

Finance Billing Analytics

Home

Dashboard

Invoices

Payments

Logout

Invoice Management

+ Create Invoice

Invoice Tracking Dashboard

Manage invoices, monitor payment status and track customer billing efficiently.

Invoice ID	Invoice No	Customer	Amount	Status	Actions
1	INV1001	Rahul Sharma	₹10000.00	Paid	<button>View</button> <button>Edit</button> <button>Delete</button>
2	INV1002	Amit Saubhari	₹20000.00	Paid	<button>View</button> <button>Edit</button> <button>Delete</button>
3	INV1003	Darshan Pandey	₹30000.00	Paid	<button>View</button> <button>Edit</button> <button>Delete</button>
4	INV1004	Abhijeet Verma	₹45000.00	Paid	<button>View</button> <button>Edit</button> <button>Delete</button>
5	INV1005	Suraj Singh	₹60000.00	Paid	<button>View</button> <button>Edit</button> <button>Delete</button>
6	INV1006	Aditya	₹70000.00	Paid	<button>View</button> <button>Edit</button> <button>Delete</button>
9	INV1007	Ravi Das	₹200000.00	Partial Paid	<button>View</button> <button>Edit</button> <button>Delete</button>
11	INV1008	aryan	₹9870.00	Pending	<button>View</button> <button>Edit</button> <button>Delete</button>

Invoice Summary

8

Total Invoices

6

Paid Invoices

1

Pending Invoices

Back to Home

Fig.31

Finance Billing Analytics

Home

Dashboard

Invoices

Payments

Logout

Create New Invoice

InvoiceNumber

INV1011

CustomerName

Aman

TotalAmount

60000.00

Status

Pending

Save Invoice

Back to Invoices

Fig.32

Finance Billing Analytics

Home

Dashboard

Invoices

Payments

Logout

Invoice Details

Invoice ID	1
Invoice Number	INV1001
Customer Name	Rahul Sharma
Invoice Date	6/1/2026
Total Amount	₹10000.00
Status	Paid

Back

Fig.33

Finance Billing Analytics

Home

Dashboard

Invoices

Payments

Logout

Edit Invoice

Invoice Number

INV1001

Customer Name

Rahul Sharma

Invoice Date

06/01/2026

Total Amount

10000.00

Status

Paid

Update Invoice

Back

Fig.34

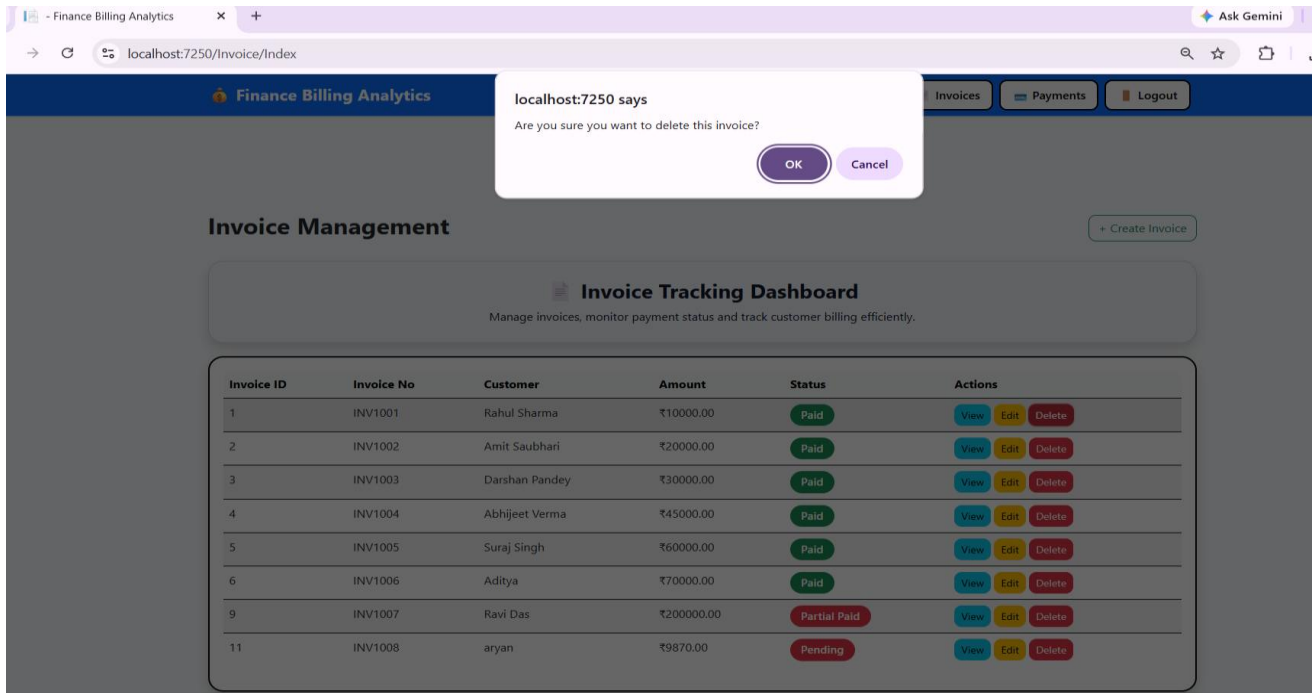


Fig.35

26.4 PAYMENT MANAGEMENT MODULE

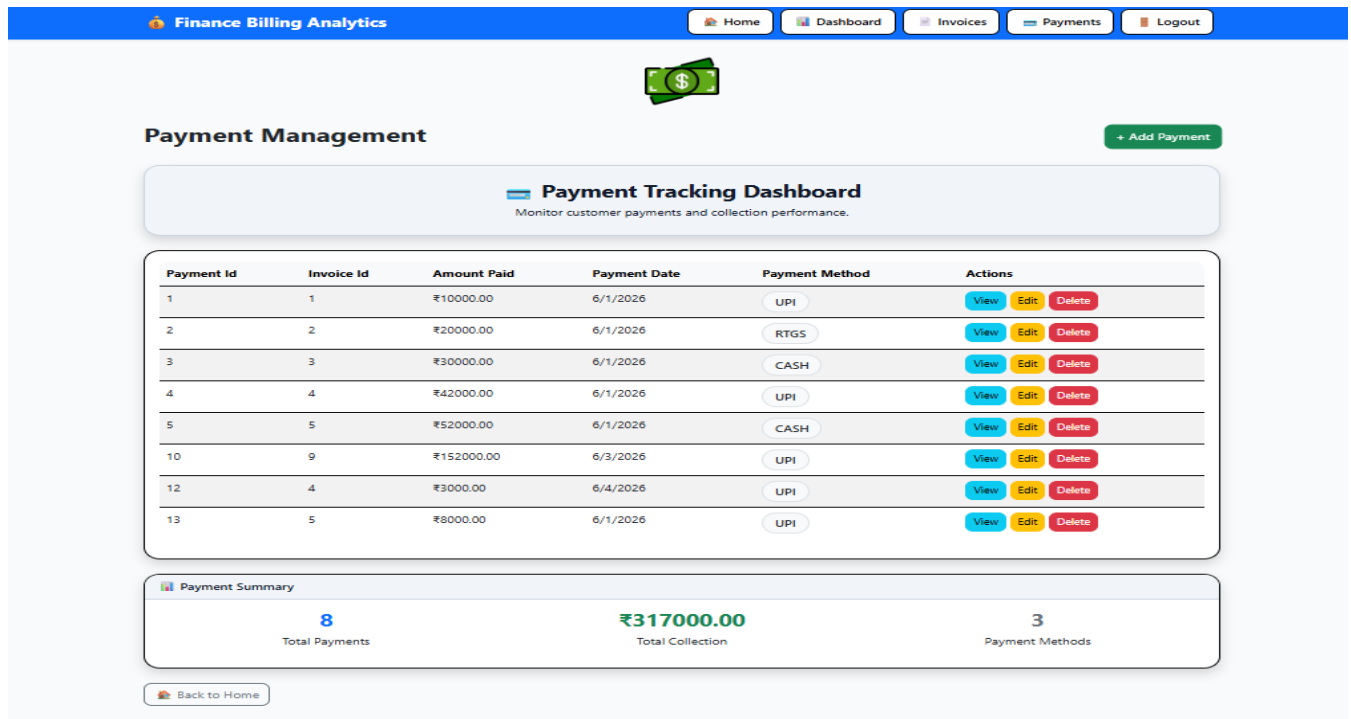


Fig.36

Finance Billing Analytics

Home

Dashboard

Invoices

Payments

Logout

Payment Details

Payment Id	1
Invoice Id	1
Amount Paid	₹10000.00
Payment Date	6/1/2026
Payment Method	UPI

← Back

Fig.37

Finance Billing Analytics

Home

Dashboard

Invoices

Payments

Logout

Add New Payment

InvoiceId

AmountPaid

PaymentDate

mm/dd/yyyy

PaymentMethod

-- Select Payment Method --

Save Payment

← Back to Payments

Fig.38

Finance Billing Analytics

Home

Dashboard

Invoices

Payments

Logout

Edit Payment

Invoice Id

1

Amount Paid

10000.00

Payment Date

06/01/2026

Payment Method

UPI

Update Payment

← Back

Fig.39

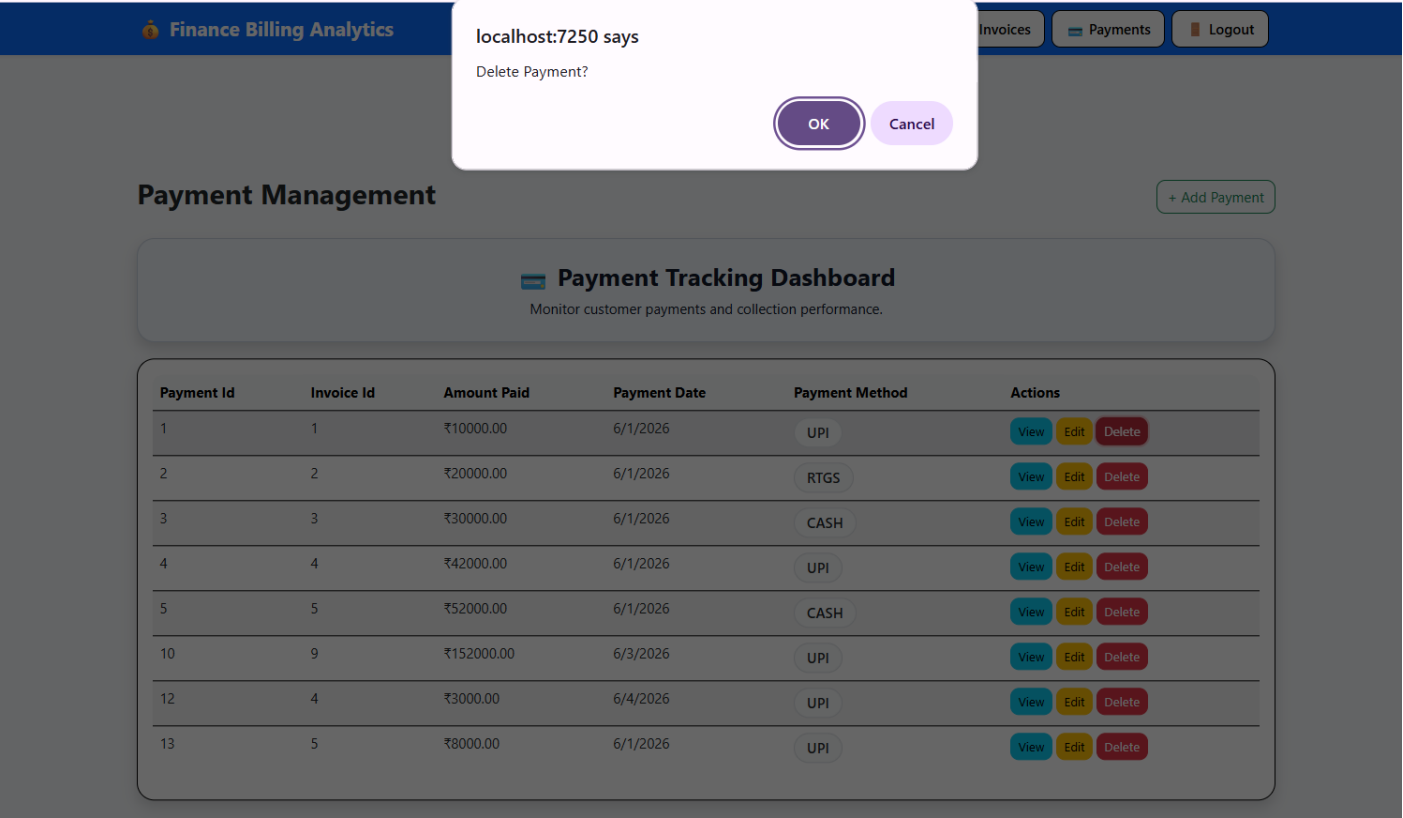


Fig.40